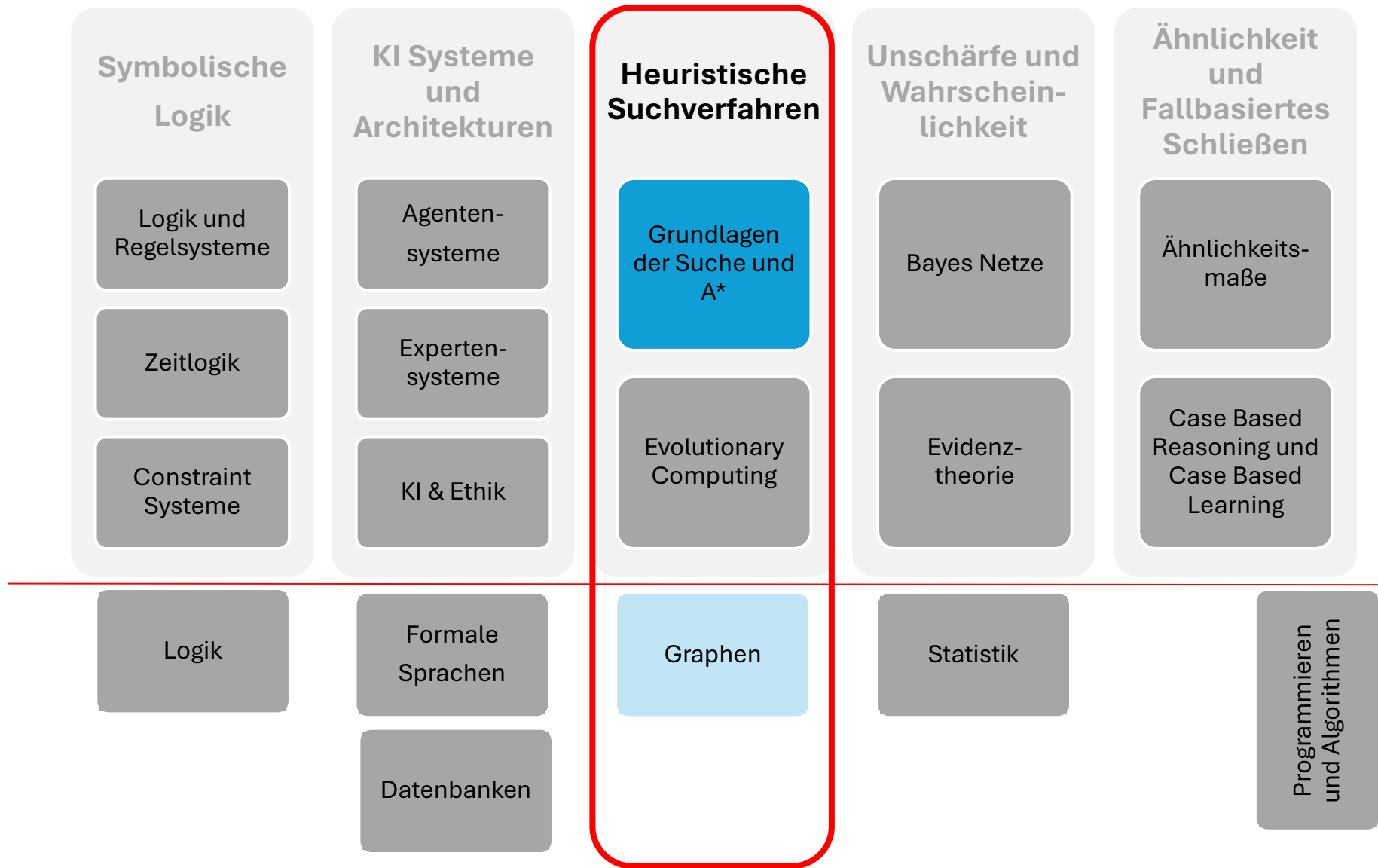


Suchverfahren in der KI

Teil 1: Grundlagen der Suche



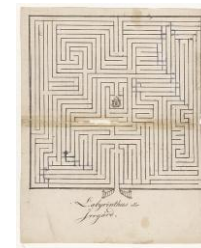


Warum Suche? Beispiel Navigation

- **Gegeben:** Start- und Endpunkt
- **Gesucht:** Optimale Route
 - Distanz
 - Reisezeit
 - Benzinverbrauch / CO₂

} via kürzeste Wege in Graphen lösbar
- Viele weitere Anwendungen!

5	3			7			
6			1	9	5		
	9	8				6	
8				6			3
4			8		3		1
7				2			6
	6					2	8
			4	1	9		5
				8		7	9

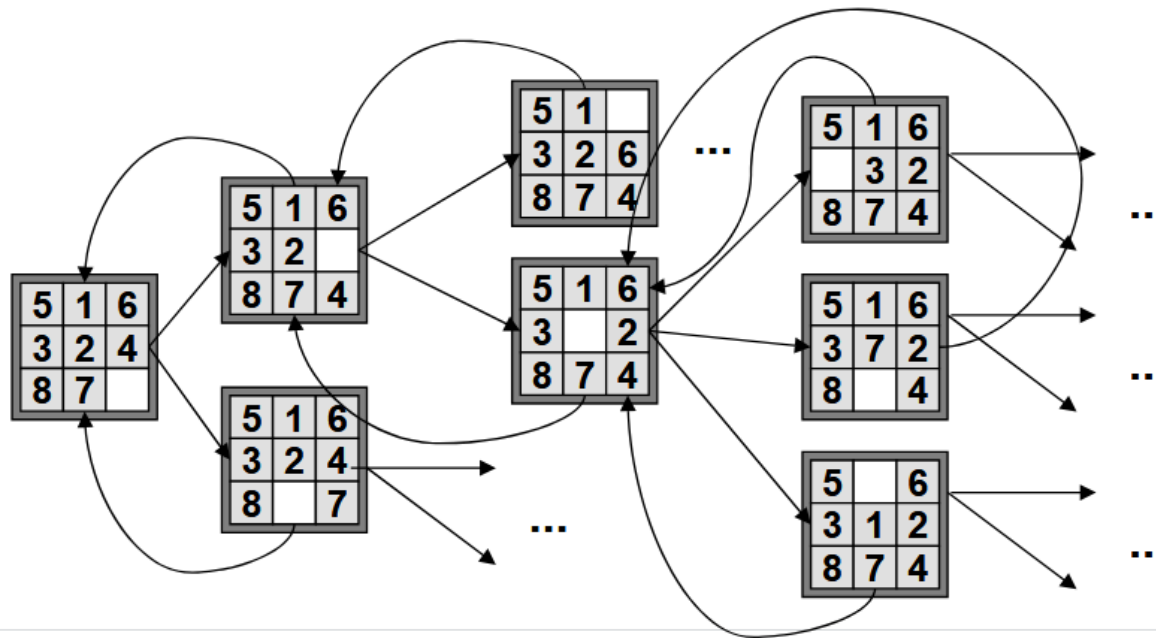
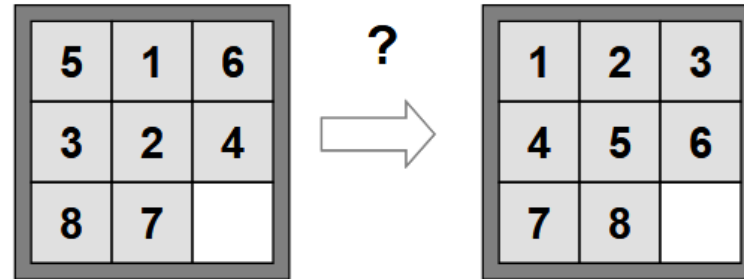


...

- **Frage:** Wie findet der Computer einen Weg im potentiell riesigen Zustandsraum?
- Brute-Force ist oft zu teuer, d.h. clevere Verfahren notwendig!

Brute-Force ist (oft) keine geeignete Lösung

Das 8-Puzzle

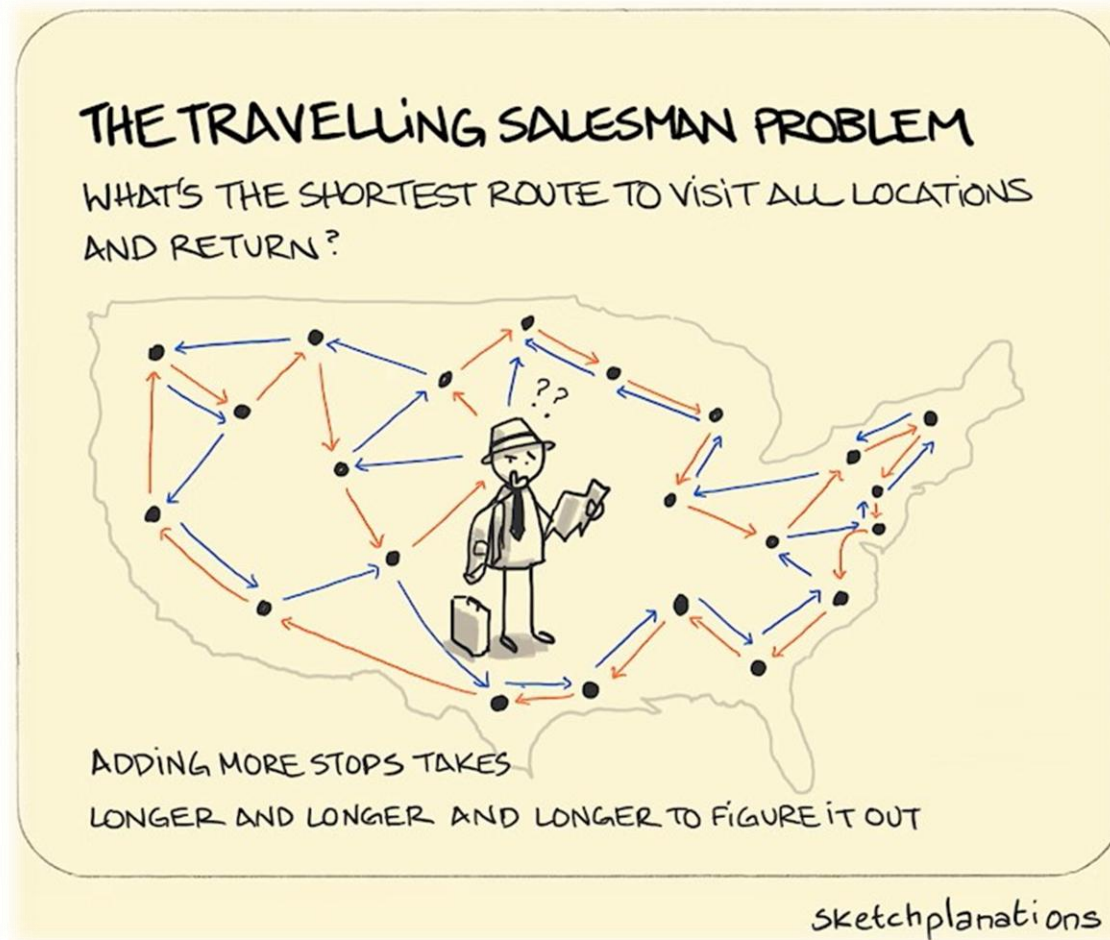


Definition (wikipedia, Zugriff: 09/25):

Ein Entscheidungsproblem ist **NP-vollständig**, wenn
1. es in **NP** liegt (Lösungen lassen sich in polynomieller Zeit verifizieren) und
2. jedes Problem in NP in polynomieller Zeit darauf reduzierbar ist.

- **Problemklasse:** NP-vollständig
- **8-Puzzle:** $9! = 362880$ Zustände
- **24-Puzzle (5x5):** $25! \approx 10^{25}$ Zustände

Brute-Force ist (oft) keine geeignete Lösung



Source: <https://sketchplanations.com/the-travelling-salesman-problem>

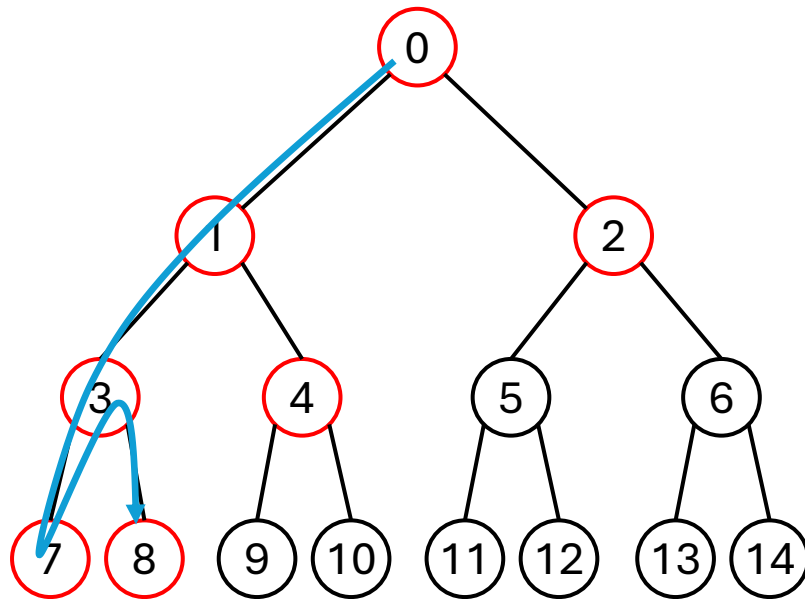
- **Problemklasse:** NP-vollständig
- $n!$ mögliche Reihenfolgen (n ist die Anzahl der Städte)
 - $(n-1)!$ wenn der Startpunkt fixiert wird
 - $(n-1)!/2$ weil die Richtung irrelevant ist
- Exakte Lösung mit vollständiger Enumeration ist ab $n > 15$ *völlig unrealistisch*

Uninformierte vs. Informierte Suche

- **Uninformierte Suchverfahren** (bekannt aus Semester 2)
 - **Tiefensuche** (Depth-First-Search, DFS)
ggf. mit maximaler Tiefe
ggf. Iterative Deepening (Start mit Tiefe 1, dann erhöhe die Tiefe falls kein Ergebnis usw.)
 - **Breitensuche** (Breadth-First-Search, BFS)
 - **Bidirektionale Suche**
Je eine BFS von Start- und Zielknoten starten bis gemeinsamer Knoten gefunden
 - „Spezialisten“: **Dijkstra** (kürzeste Wege in Graphen)

Erinnerung: Tiefensuche und Breitensuche

Tiefensuche:



Zielknoten

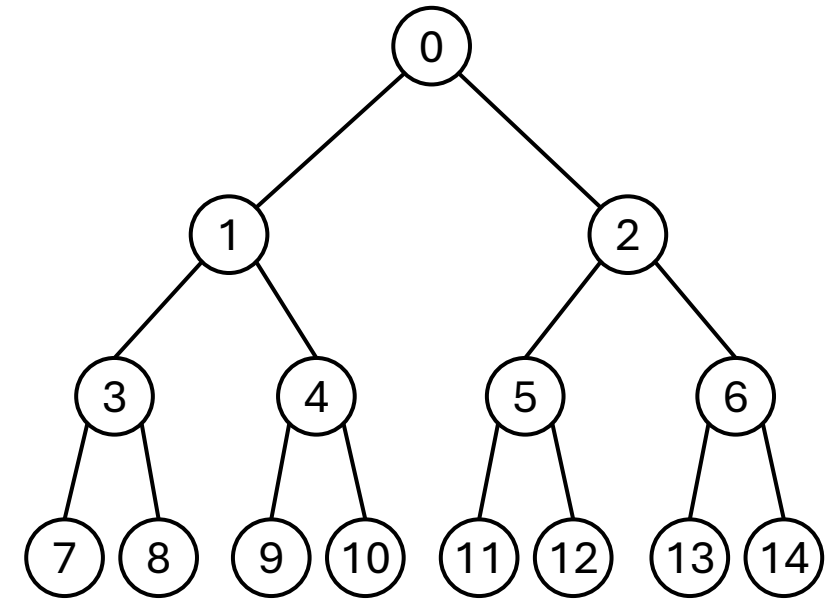
Rekursiver Algorithmus: DFS(v)

Markiere (v)

Für alle w mit $(v, w) \in E$:

Wenn w unmarkiert, dann DFS(w)

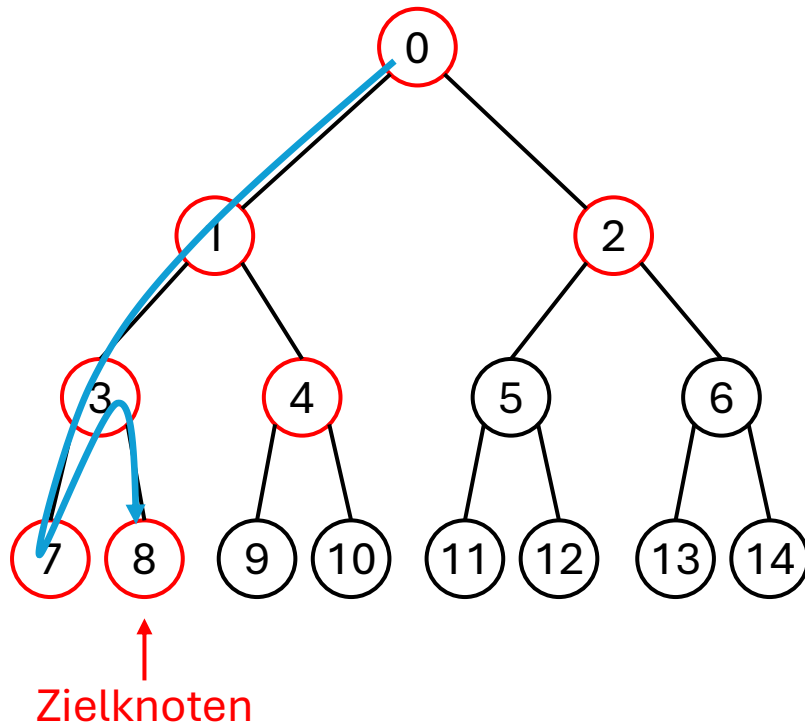
Breitensuche:



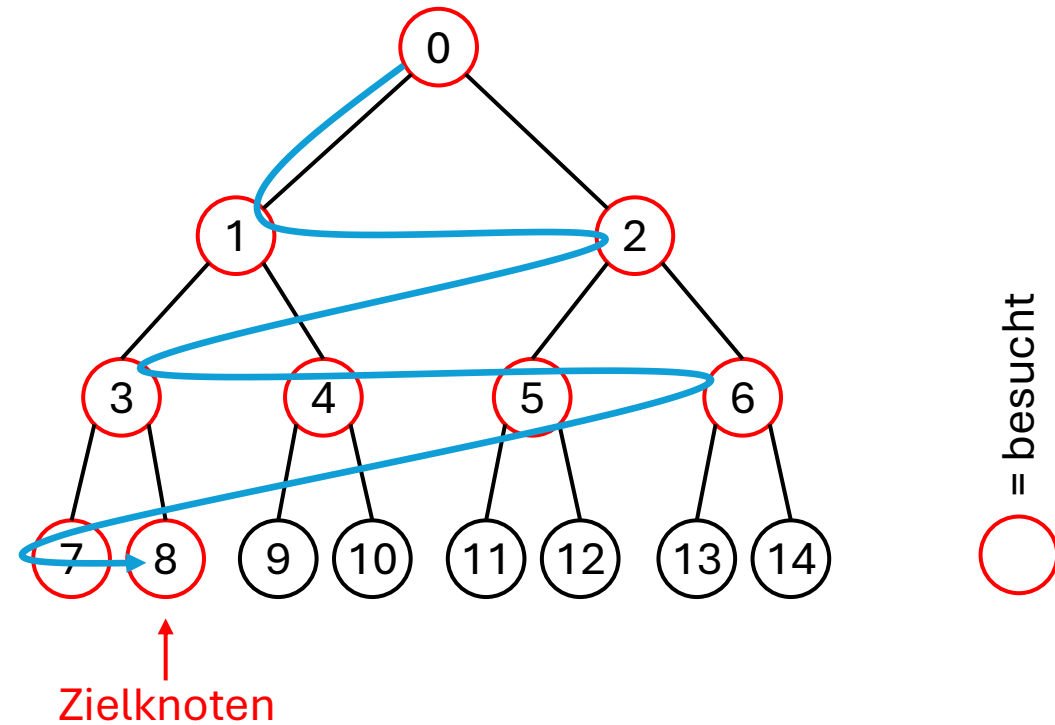
 = besucht

Erinnerung: Tiefensuche und Breitensuche

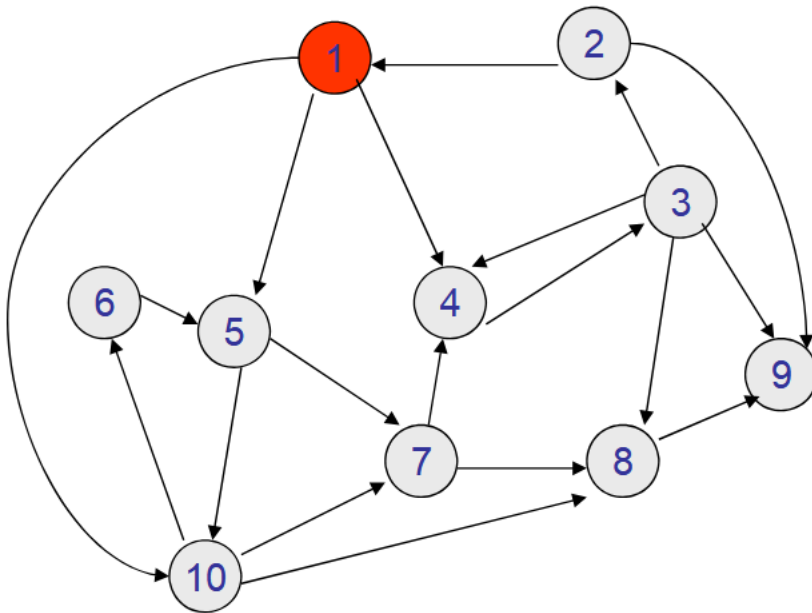
Tiefensuche:



Breitensuche:



Tiefensuche: Stack als Datenstruktur (iterativ)

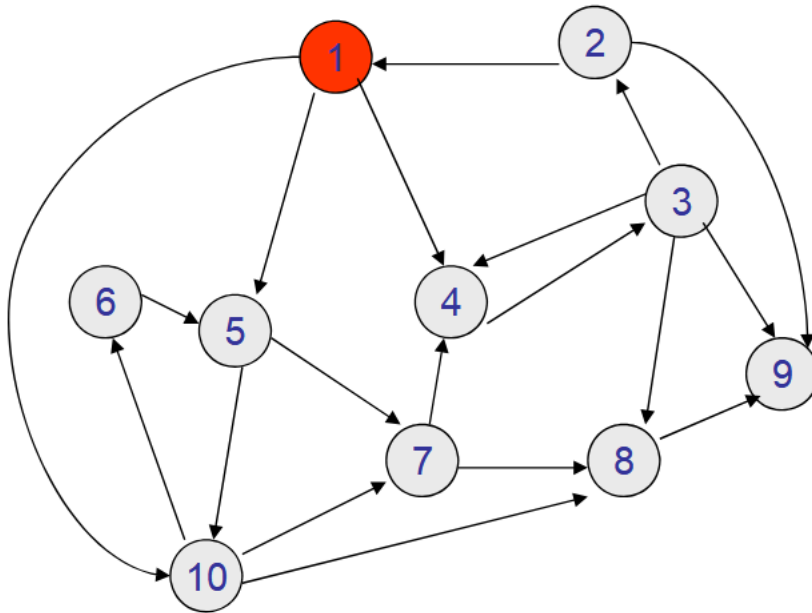


Stack: (1)	Visited ()
Stack: (4 5 10)	Visited (1)
Stack: (3 5 10)	Visited (1 4)
Stack: (2 8 9 5 10)	Visited (1 4 3)
Stack: (9 8 9 5 10)	Visited (1 4 3 2)
Stack: (8 9 5 10)	Visited (1 4 3 2 9)
Stack: (5 10)	Visited (1 4 3 2 9 8)
Stack: (7 10 10)	Visited (1 4 3 2 9 8 5)
Stack: (10 10)	Visited (1 4 3 2 9 8 5 7)
Stack: (6 10)	Visited (1 4 3 2 9 8 5 7 10)
Stack: ()	Visited (1 4 3 2 8 9 5 7 10 6)

Speichert, in welchem Knoten DFS gerade ist und welche Knoten nach dem Zurückspringen als nächstes kommen

Menge aller bereits besuchten Knoten (Vermeidung von Zyklen)

Breitensuche: Queue als Datenstruktur (iterativ)



Noch zu expandierende Knoten

OPEN: (1)

OPEN: (4 5 10)

OPEN: (5 10 3)

OPEN: (10 3 7)

OPEN: (3 7 6 8)

OPEN: (7 6 8 2 9)

OPEN: (6 8 2 9)

OPEN: (8 2 9)

OPEN: (2 9)

OPEN: (9)

OPEN: ()

CLOSED ()

CLOSED (1)

CLOSED (1 4)

CLOSED (1 4 5)

CLOSED (1 4 5 10)

CLOSED (1 4 5 10 3)

CLOSED (1 4 5 10 3 7)

CLOSED (1 4 5 10 3 7 6)

CLOSED (1 4 5 10 3 7 6 8)

CLOSED (1 4 5 10 3 7 6 8 2)

CLOSED (1 4 5 10 3 7 6 8 2 9)

Bereits expandierte, d.h. vollständig
verarbeitete, Knoten

Uninformierte vs. Informierte Suche

- **Uninformierte Suchverfahren** (bekannt aus Semester 2)

- **Tiefensuche** (Depth-First-Search, DFS)

- ggf. mit maximaler Tiefe

- ggf. Iterative Deepening (Start mit Tiefe 1, dann erhöhe die Tiefe falls kein Ergebnis)

- **Breitensuche** (Breadth-First-Search, BFS)

- **Bidirektionale Suche**

- Je eine BFS von Start- und Zielknoten starten bis gemeinsamer Knoten gefunden

- „Spezialisten“: **Dijkstra** (kürzeste Wege in Graphen)

- **Informierte (heuristische & statistische) Suchverfahren**
(Fokus heute)

- **Best-First-Search**

- **A***

- **Kennen** nur den Raum der möglichen Zustände (Knoten), die möglichen Zustandsübergänge (Kanten) und die Optionen im aktuellen Zustand (erreichbare Nachbarknoten)
- **Haben keine Ahnung**, wo der Zielzustand liegt

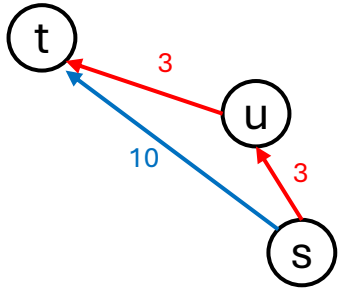
- Haben **zusätzliches Problemwissen** über die Zielrichtung (z.B. eine Heuristik)
- Nutzen diese, um **vielversprechende Zustände zu bevorzugen** und zu erweitern

Suchverfahren in der KI

Teil 2: Wiederholung: Dijkstra (uniformiert)



Einführen am Beispiel: Kürzeste Wege in Graphen



$\langle s, t \rangle$
 $\langle s, u, t \rangle$

$$w(\langle s, t \rangle) = 10$$

$$w(\langle s, u, t \rangle) = 3 + 3 = 6$$

$\langle s, u, t \rangle$

- Gegeben sei ein **gerichteter, gewichteter Graph** $G = (V, E, w)$ mit
 - einer endlichen Menge von **Knoten** V
 - einer Menge von **Kanten** $E \subseteq \{(u, v) \mid u, v \in V\}$ *Ungerichtet: $\{u, v\}$*
 - und **Kantengewichten** $w: E \rightarrow \mathbb{R}$

- Ein **Weg** (Pfad) $p_{s,t}$ der Länge k von einem Startknoten $s \in V$ zu einem Zielknoten $t \in V$ ist eine Folge $p_{s,t} = \langle v_0, v_1, v_2, \dots, v_k \rangle$ von Knoten, für die gilt:

$$v_0 = s, \quad v_k = t$$

$$(v_i, v_{i+1}) \in E \text{ für alle } 0 \leq i \leq k-1$$

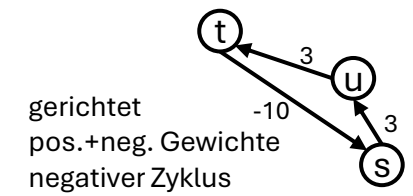
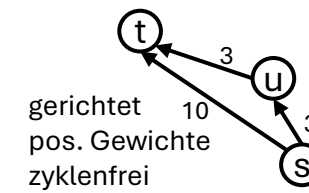
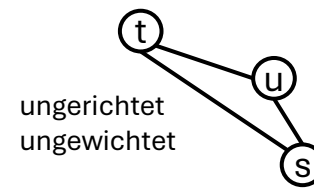
- Das **Gewicht eines Weges** $p_{s,t} = \langle v_0, v_1, v_2, \dots, v_k \rangle$ ist die Summe seiner Kantengewichte

$$w(p) = \sum_{i=1}^k w(v_{i-1}, v_i)$$

- Ein Weg $p_{s,t}$ mit Gewicht $w(p_{s,t})$ heißt **kürzester Weg** von s nach t falls gilt

$$w(p_{s,t}) = \min \{w(\tilde{p}_{s,t}) \mid \tilde{p}_{s,t} \text{ ist ein Pfad von } s \text{ nach } t\}$$

Varianten



A: Single Source Shortest Path

- Finde die kürzesten Wege zwischen **einem Startknoten** s und **allen** anderen Knoten
- Verschiedene **Algorithmen**, Wahl gemäß Eigenschaften von G :

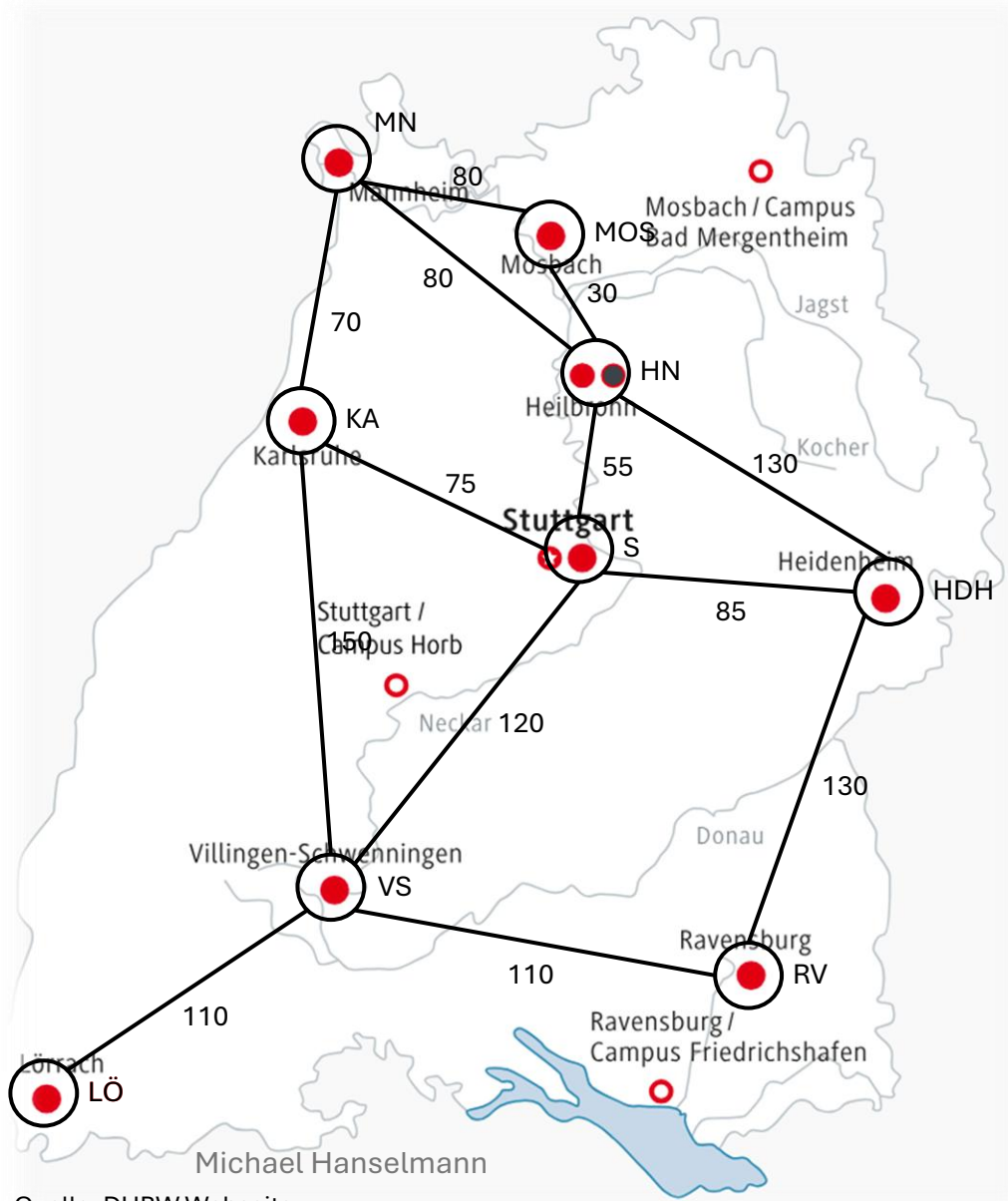
	gerichtet / ungerichtet	Gewichte	Zyklenfrei	Komplexität Zeit / Speicher	
Breitensuche	beides	keine	egal	$O(V + E)$	$O(V)$
Bellman-Ford	gerichtet	pos.+neg.	nicht- neg.Zyk.	$O(V * E)$	$O(V)$
Topol. Suche	gerichtet	pos.+neg.	zyklenfrei	$O(V + E)$	$O(V)$
Dijkstra	beides	positiv	egal	$O(E * \log(V))$	$O(V)$
A* (1 Zielknoten)					

B: All Pairs Shortest Path

- Finde kürzeste Wege zwischen **allen Knotenpaaren**
- Verschiedene **Algorithmen**, Wahl gemäß Eigenschaften von G :

	gerichtet / ungerichtet	Gewichte	Zyklenfrei	Komplexität Zeit / Speicher	
Floyd-Warshall	ungerichtet	positiv	egal	$O(V ^3)$	$O(V ^2)$
	gerichtet	pos.+neg.	nicht- neg.Zyklen		
Johnson	gerichtet	pos.+neg.	nicht- neg.Zyklen	$O(V ^2 * \log(V) + V * E)$	

Dijkstra Beispiel



1959

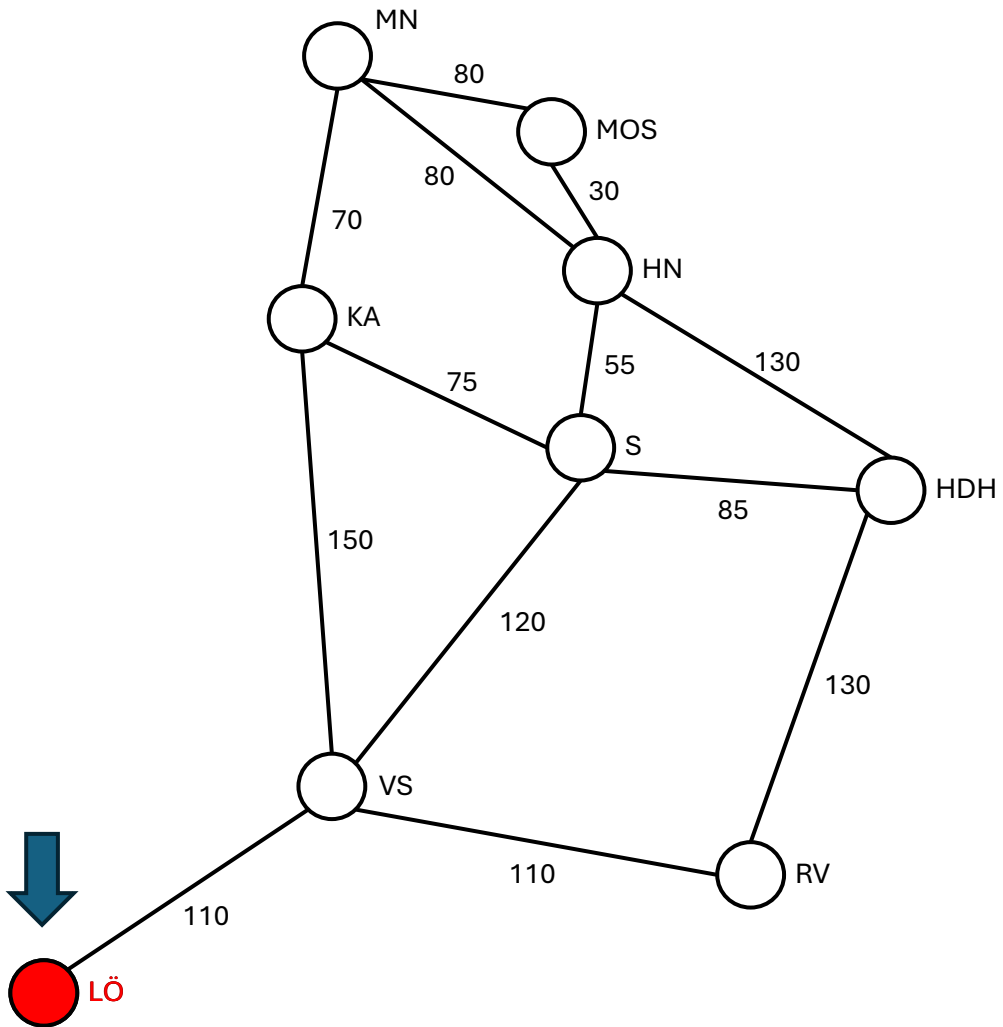
Gegeben:

- Ungerichteter Graph mit 9 **DHBW** Standorten
- Kanten mit Gewichten (Distanz in km)

Aufgabe:

- Finde kürzeste Wege von Startknoten Lörrach ($s = LÖ$) zu den anderen Standorten

Dijkstra Beispiel: Offline



Dijkstra Beispiel

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	∞	∞	∞	0	∞	∞	∞	∞	∞
$\text{prev}[v]$	-	-	-	-	-	-	-	-	-

(Aktuelle) Weglänge von Startknoten
 $s=\text{LÖ}$ zu Knoten v , d.h.
 $\text{dist}[v] = w(p_{s,v})$

Vorgängerknoten
von v auf (aktuell)
kürzesten Weg

• Initialisiere:

$Q = []$

$Q.\text{addWithPriority}(\text{LÖ}, 0)$

for each vertex v in V :

$\text{dist}[v] \leftarrow \infty$

$\text{prev}[v] \leftarrow \text{'-'}$

$\text{dist}[\text{LÖ}] \leftarrow 0$

// Prioritätsliste

// Füge Startknoten zu

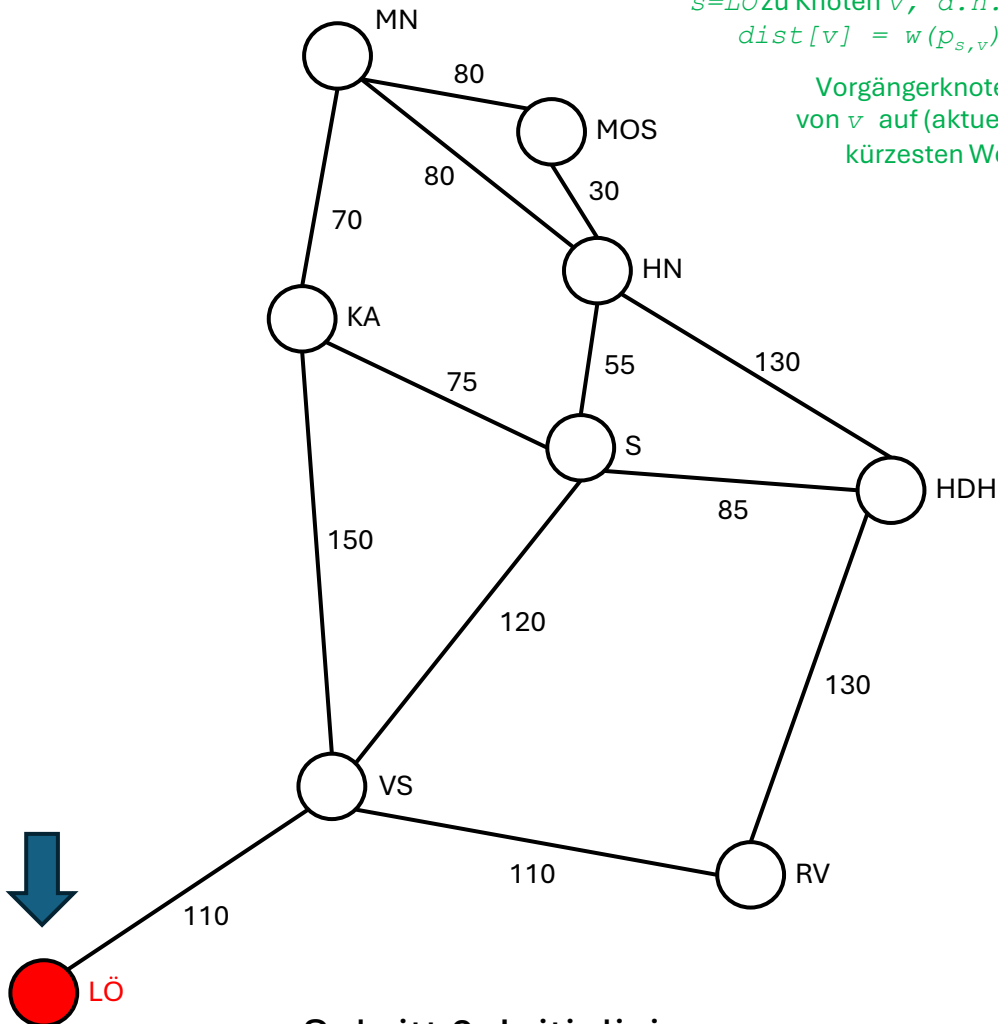
// Kandidatenliste hinzu

// Distanz vom Startknoten

// (∞ da bislang kein Weg gefunden)

// kein Vorgängerknoten

// Distanz zum Startknoten



Schritt 0: Initialisiere

Dijkstra Beispiel

Knotenarten:



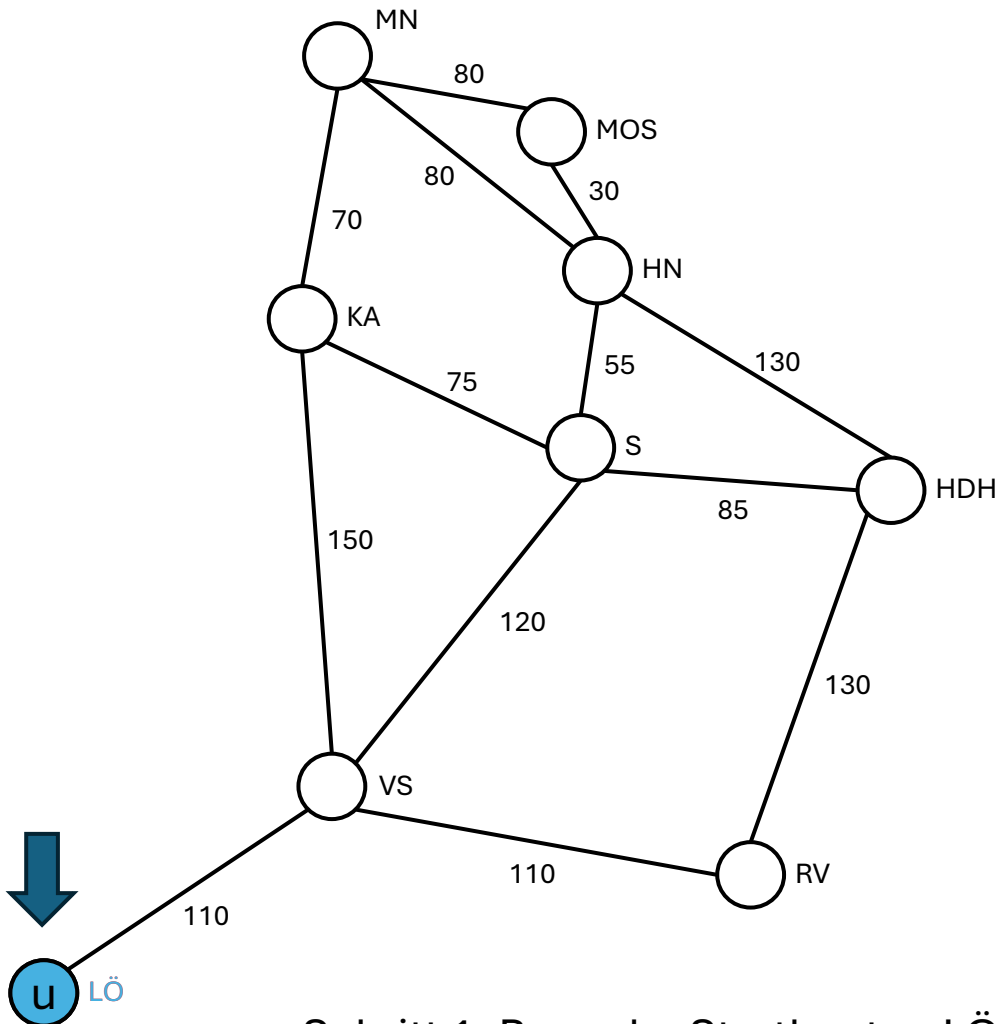
Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



Schritt 1: Besuche Startknoten LÖ

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	∞	∞	∞	0	∞	∞	∞	∞	∞
$\text{prev}[v]$	-	-	-	-	-	-	-	-	-

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	∞	∞	∞	0	∞	∞	∞	∞	110
$\text{prev}[v]$	-	-	-	-	-	-	-	-	LO

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

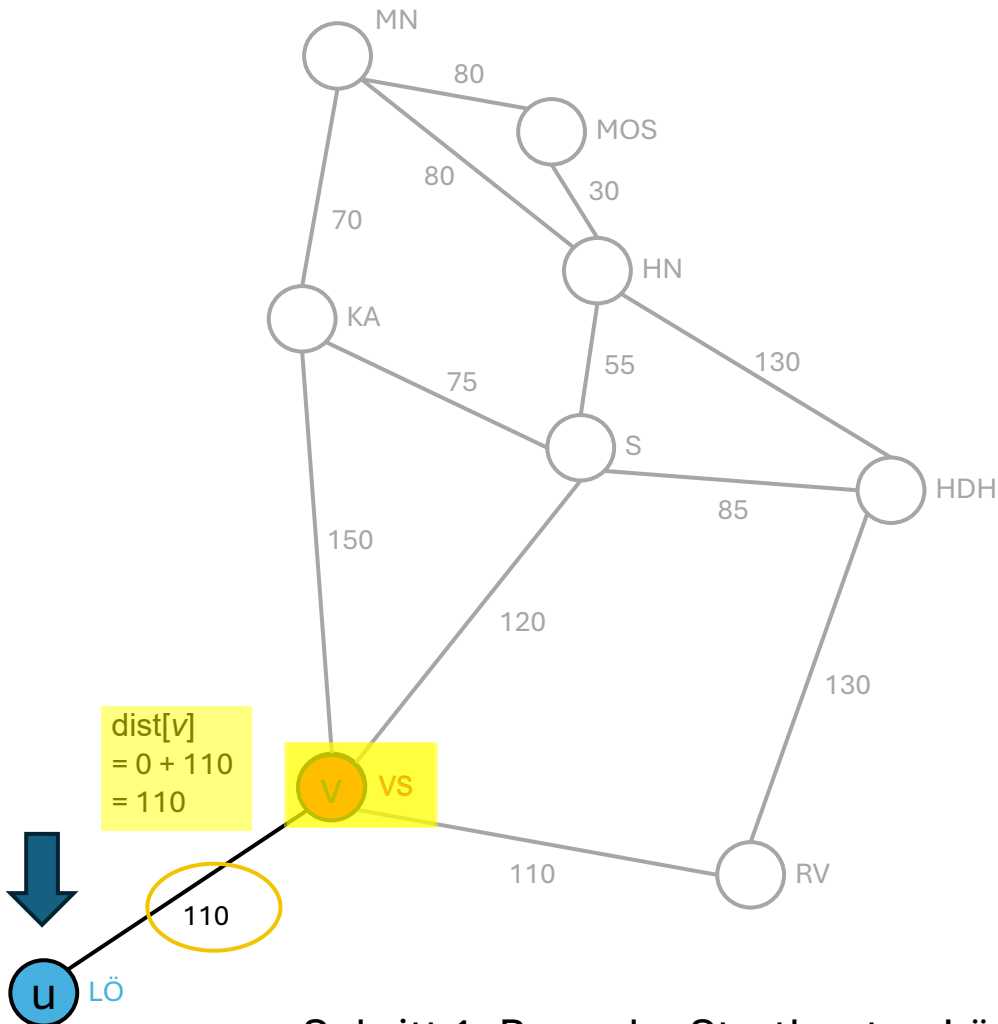
$Q.\text{addWithPriority}(v, \text{dist}[v])$

elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$



Schritt 1: Besuche Startknoten Lörrach

Dijkstra Beispiel

Knotenarten:



Bereits besucht



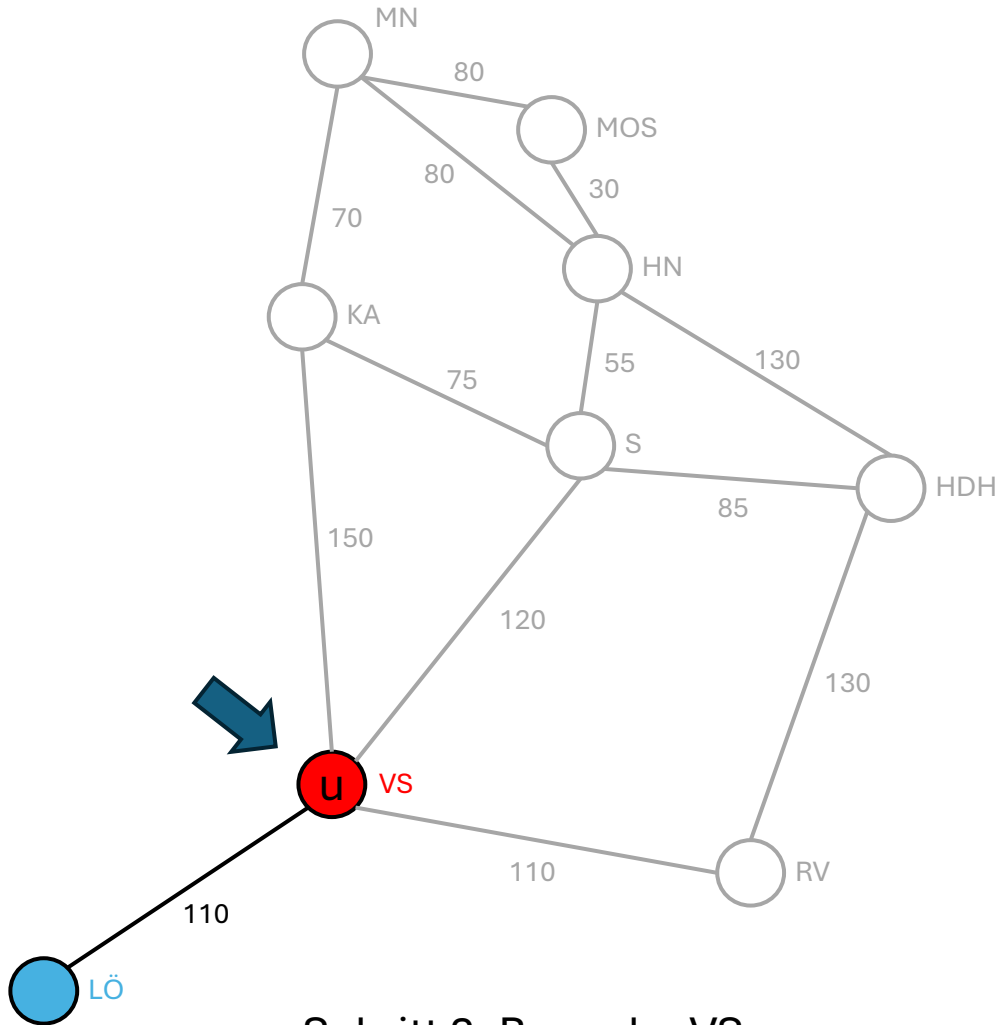
Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	∞	∞	∞	0	∞	∞	∞	∞	110
$\text{prev}[v]$	-	-	-	-	-	-	-	-	LO



Schritt 2: Besuche VS

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

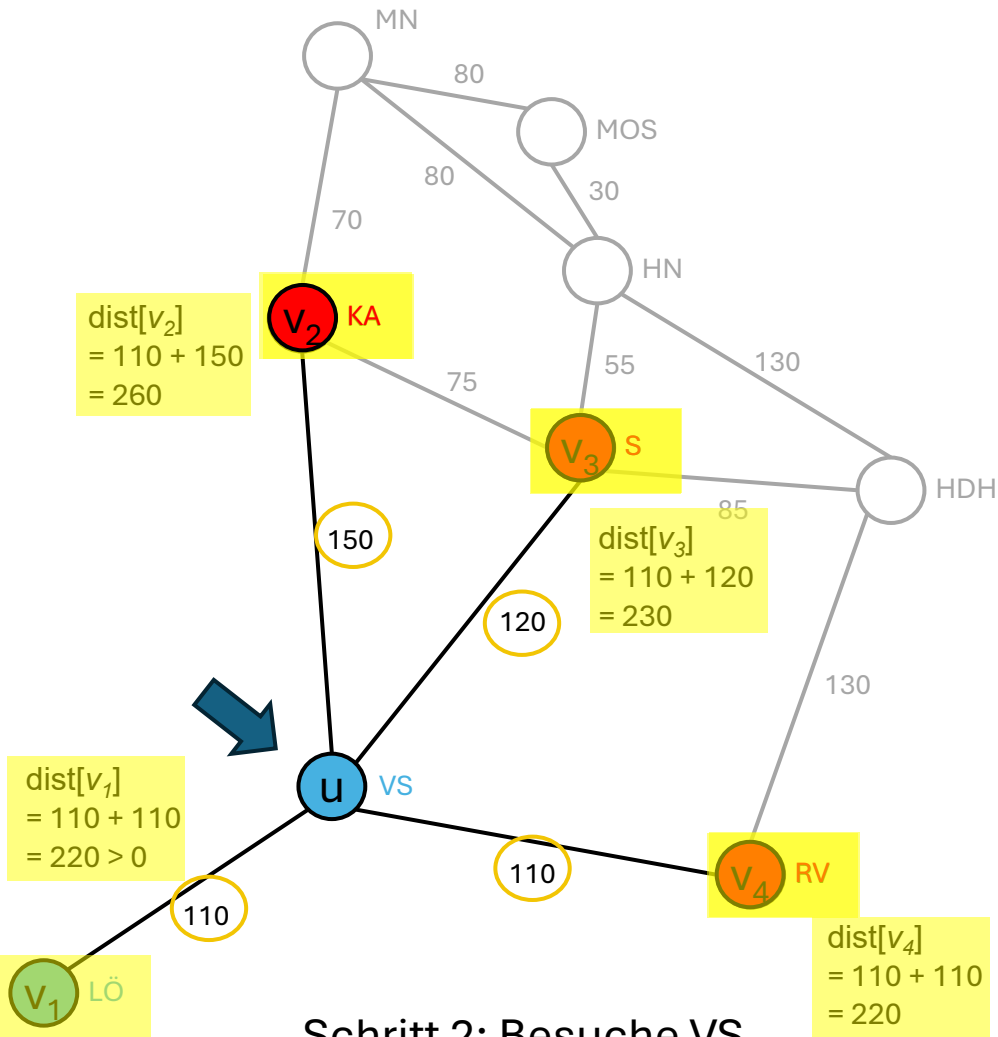
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 2: Besuche VS

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
dist[v]	∞	∞	260	0	∞	∞	220	230	110
prev[v]	-	-	VS	-	-	-	VS	VS	LO

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if dist[v] == ∞ : // falls bislang nie besucht

prev[v] $\leftarrow u$ // Setze u als Vorgänger

dist[v] $\leftarrow$ dist[u] + $w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

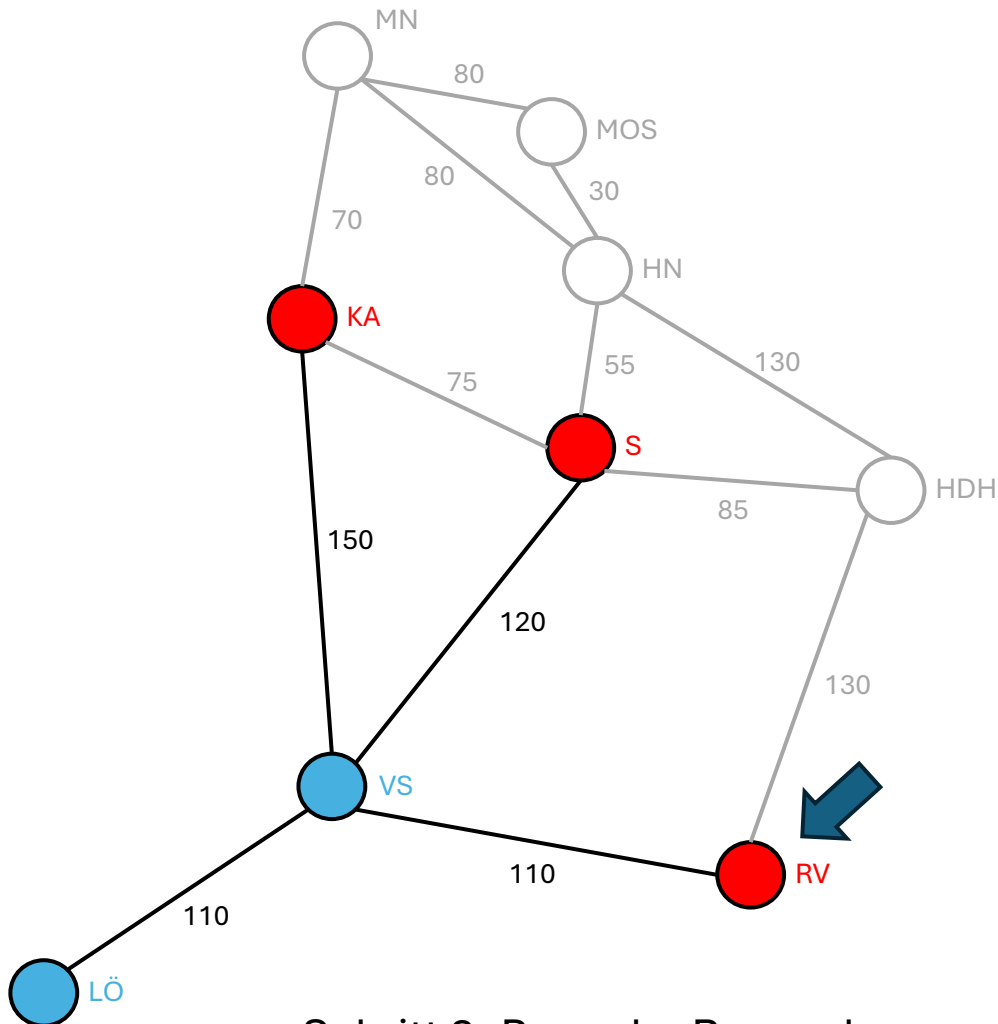
elseif dist[u] + $w(u, v)$ < dist[v]: // falls neue kleinere Distanz

prev[v] $\leftarrow u$ // Setze u als Vorgänger

dist[v] $\leftarrow$ dist[u] + $w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 3: Besuche Ravensburg

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	∞	∞	260	0	∞	∞	220	230	110
$\text{prev}[v]$	-	-	VS	-	-	-	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

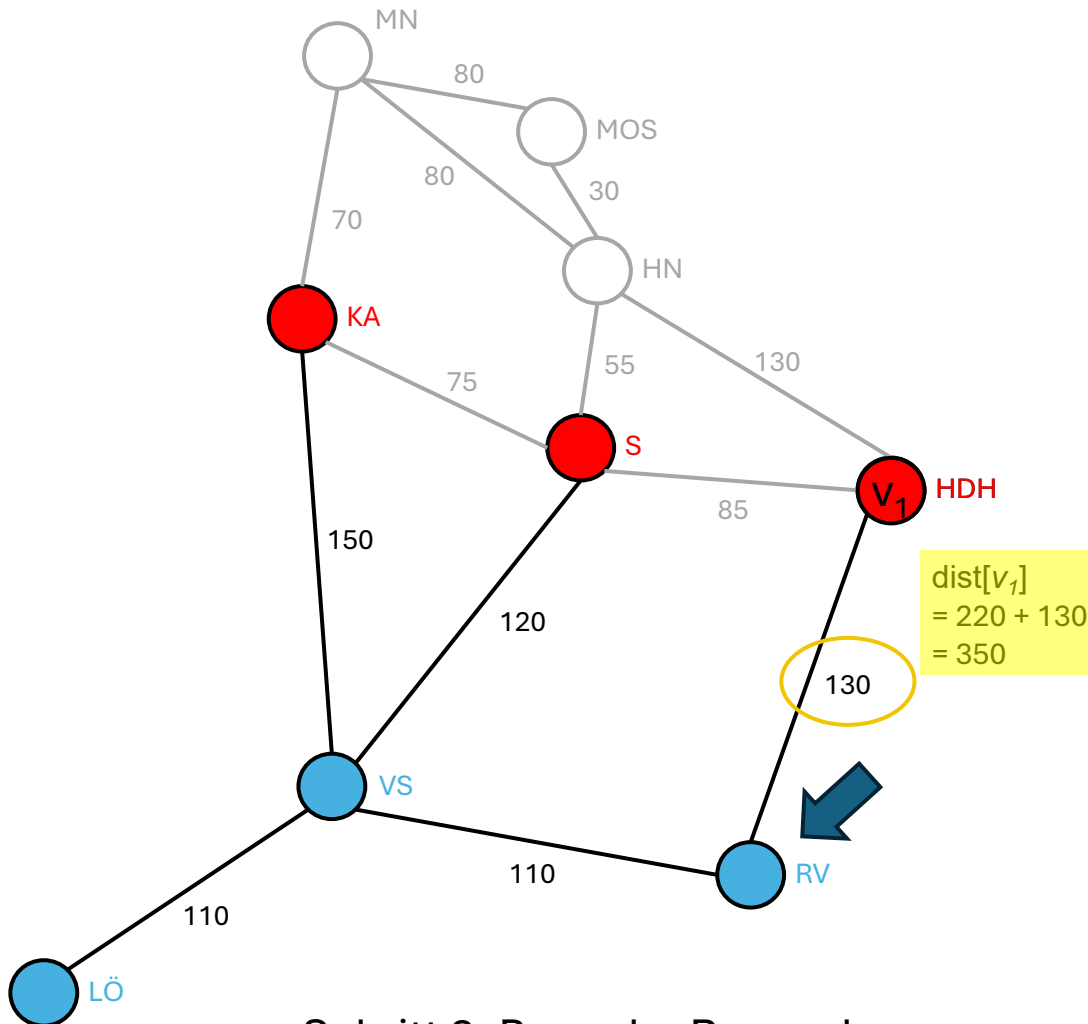
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 3: Besuche Ravensburg

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	350	∞	260	0	∞	∞	220	230	110
$\text{prev}[v]$	RV	-	VS	-	-	-	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

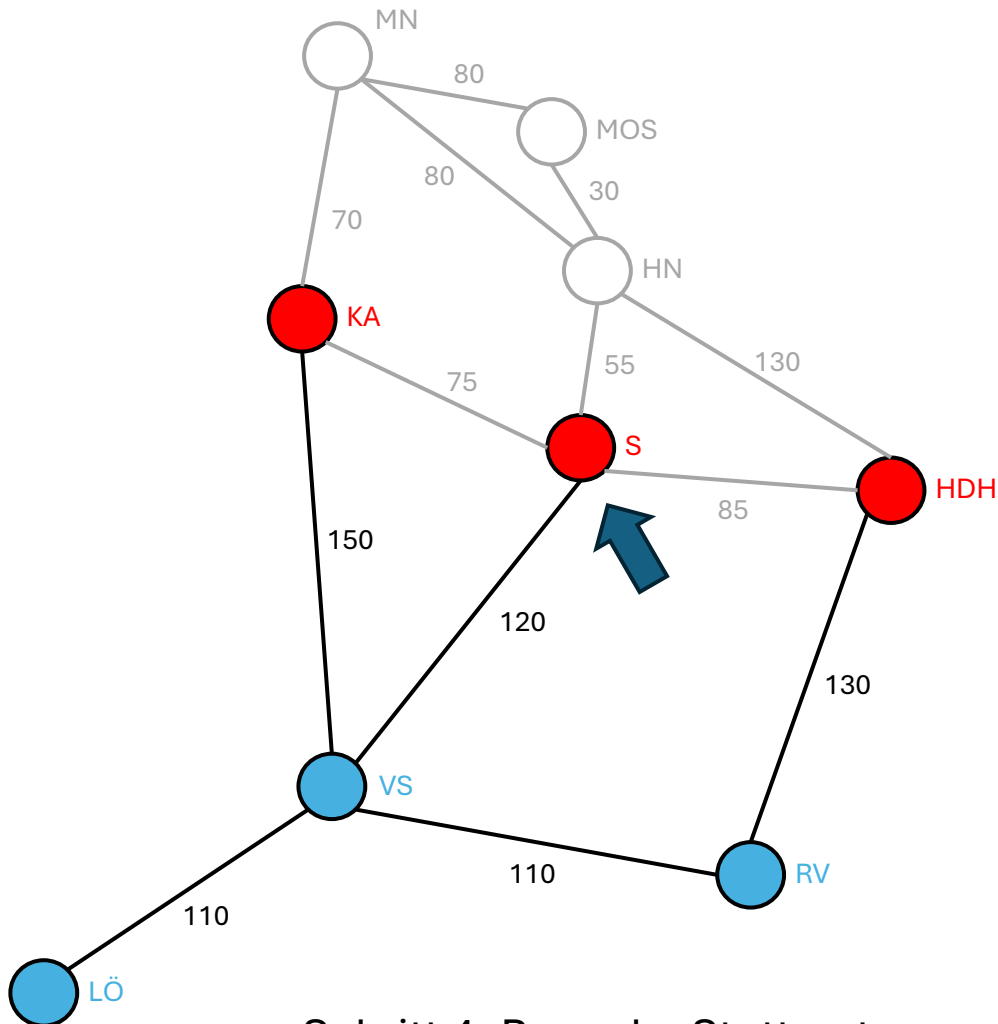
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 4: Besuche Stuttgart

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	350	∞	260	0	∞	∞	220	230	110
$\text{prev}[v]$	RV	-	VS	-	-	-	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

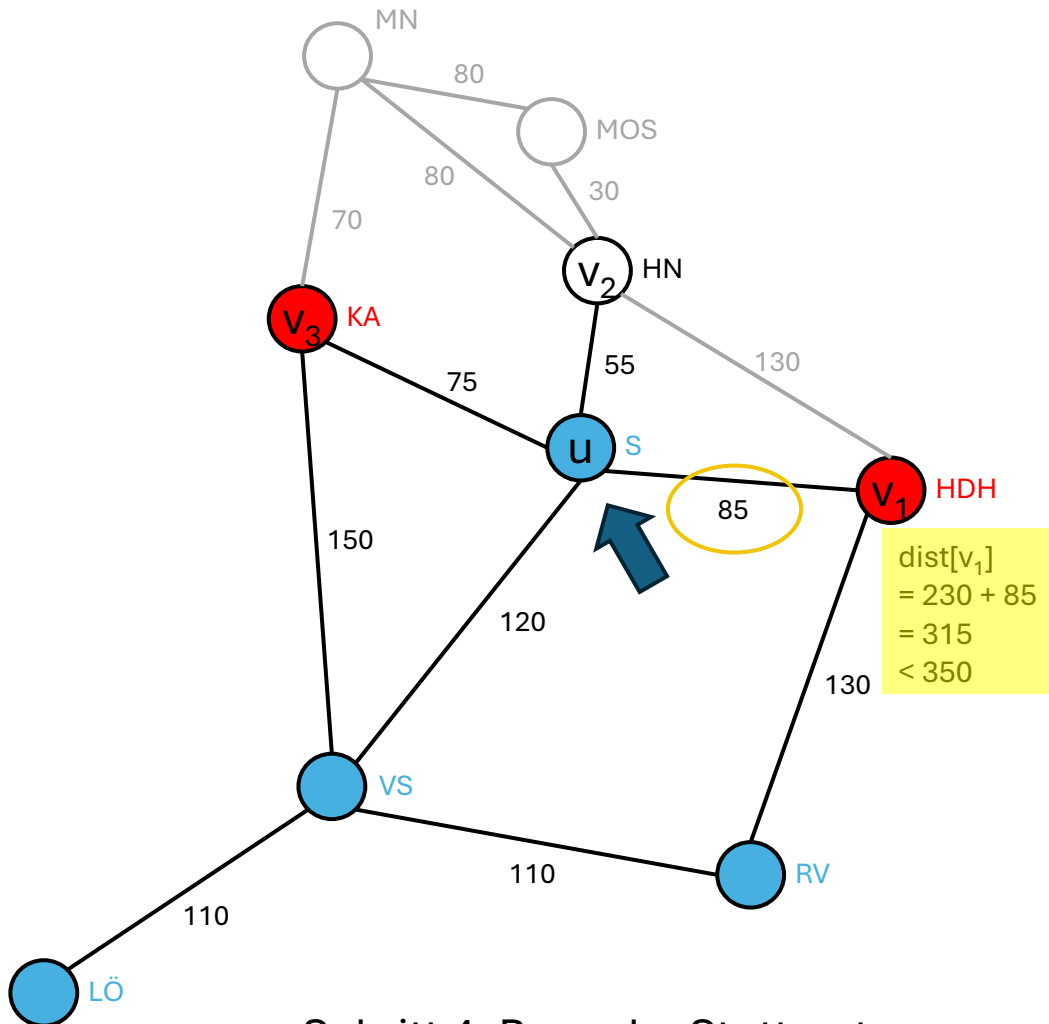
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 4: Besuche Stuttgart

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	350	∞	260	0	∞	∞	220	230	110
$\text{prev}[v]$	RV	-	VS	-	-	-	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

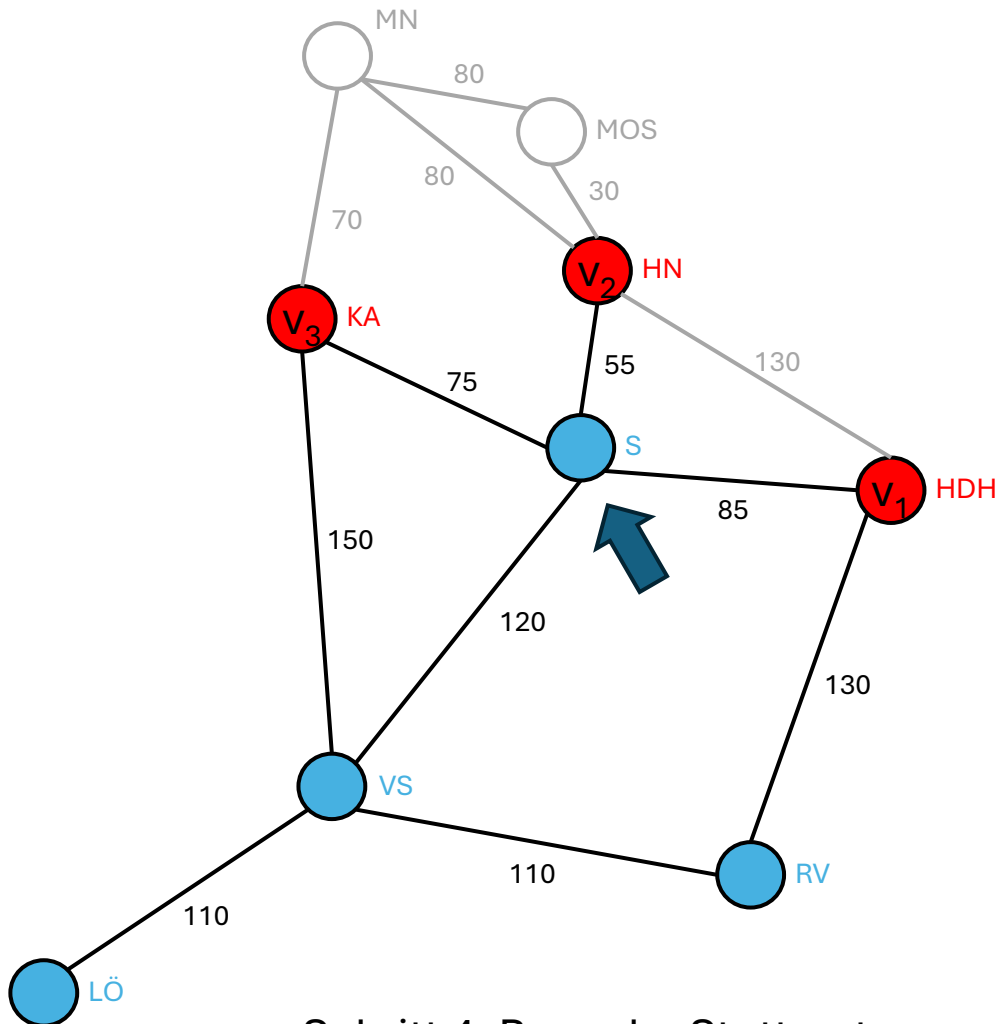
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 4: Besuche Stuttgart

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	315	285	260	0	∞	∞	220	230	110
$\text{prev}[v]$	S	S	VS	-	-	-	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

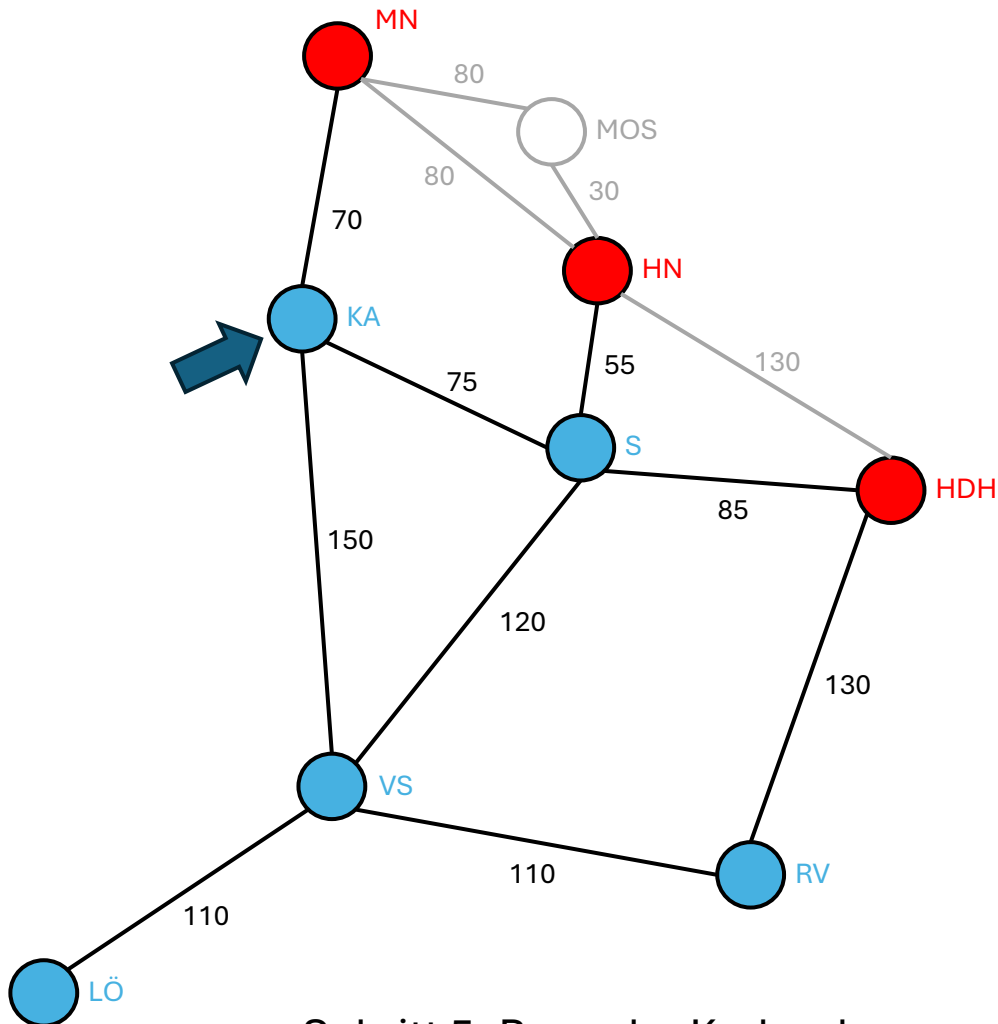
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 5: Besuche Karlsruhe

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	315	285	260	0	330	∞	220	230	110
$\text{prev}[v]$	S	S	VS	-	KA	-	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

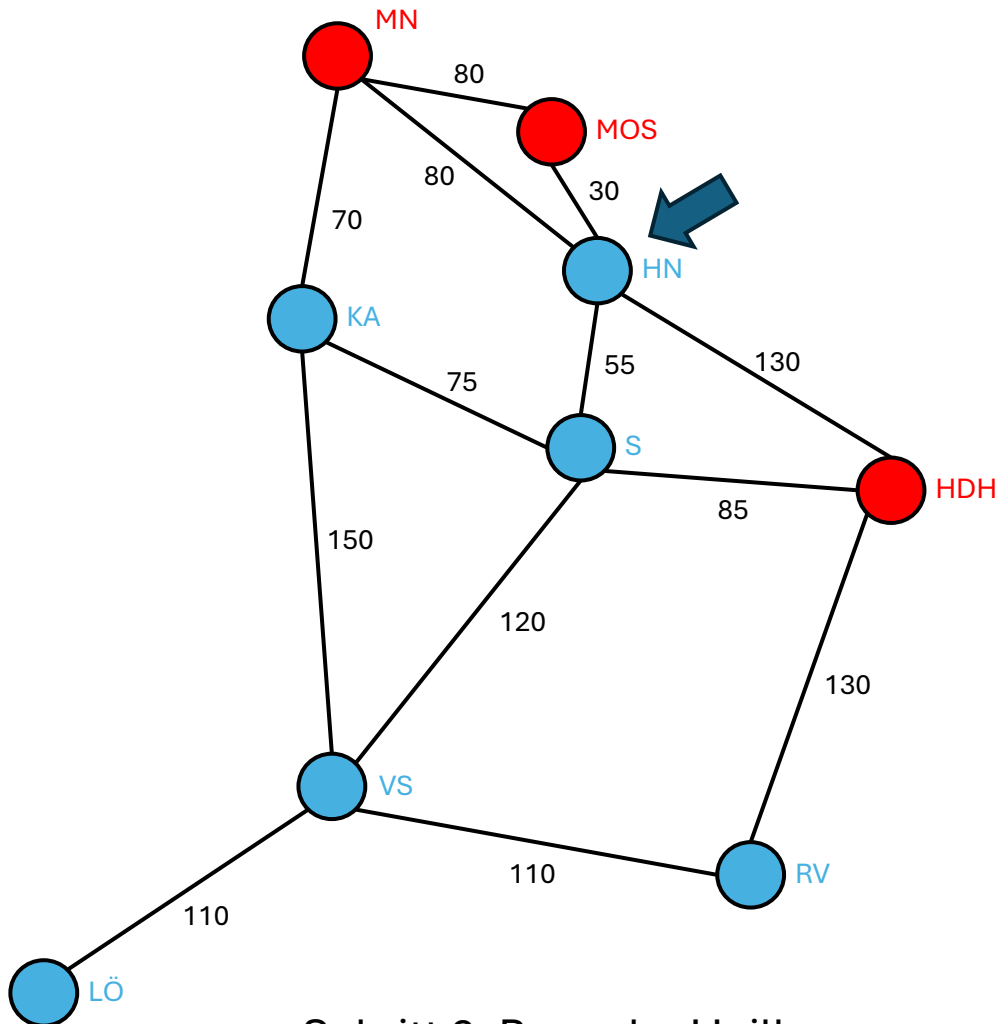
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 6: Besuche Heilbronn

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	315	285	260	0	330	315	220	230	110
$\text{prev}[v]$	S	S	VS	-	KA	HN	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

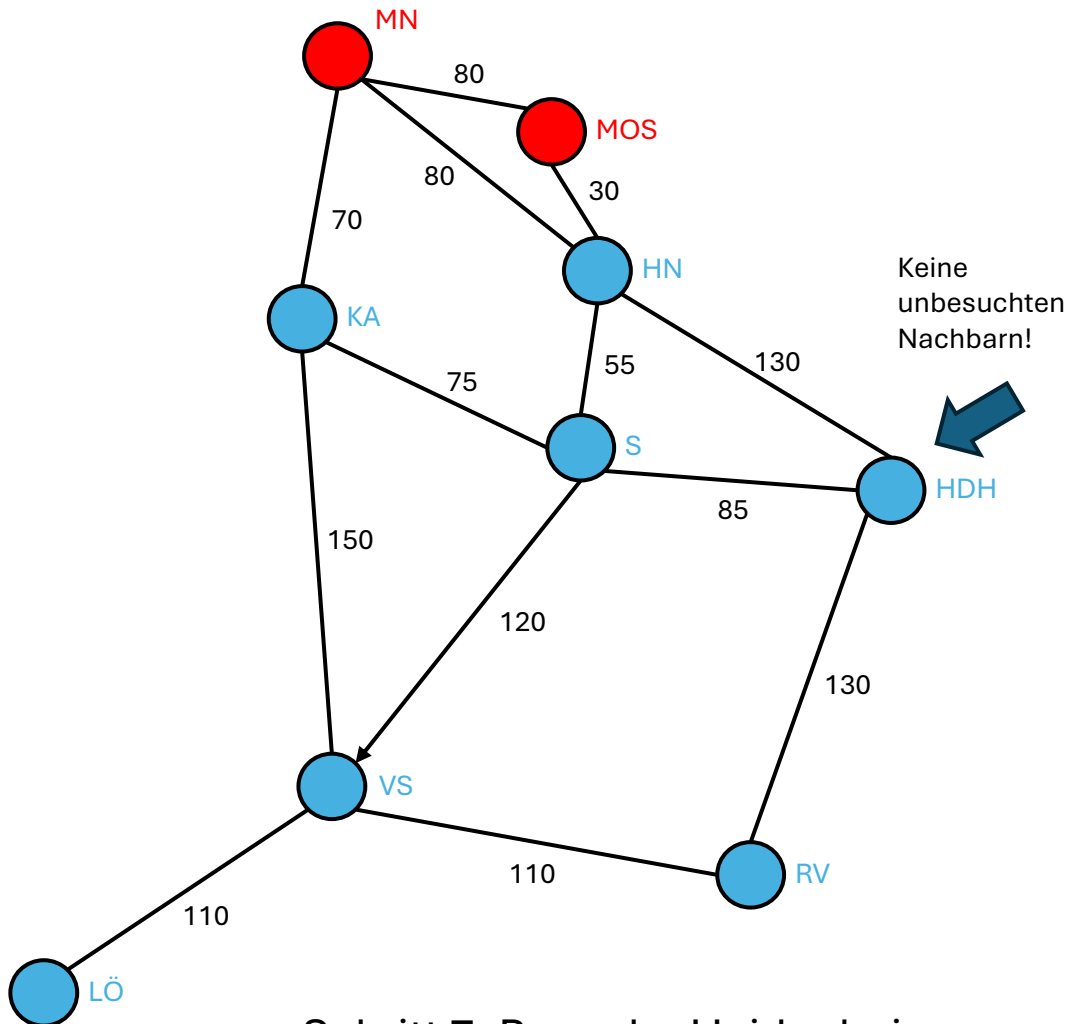
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 7: Besuche Heidenheim

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	315	285	260	0	330	315	220	230	110
$\text{prev}[v]$	S	S	VS	-	KA	HN	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

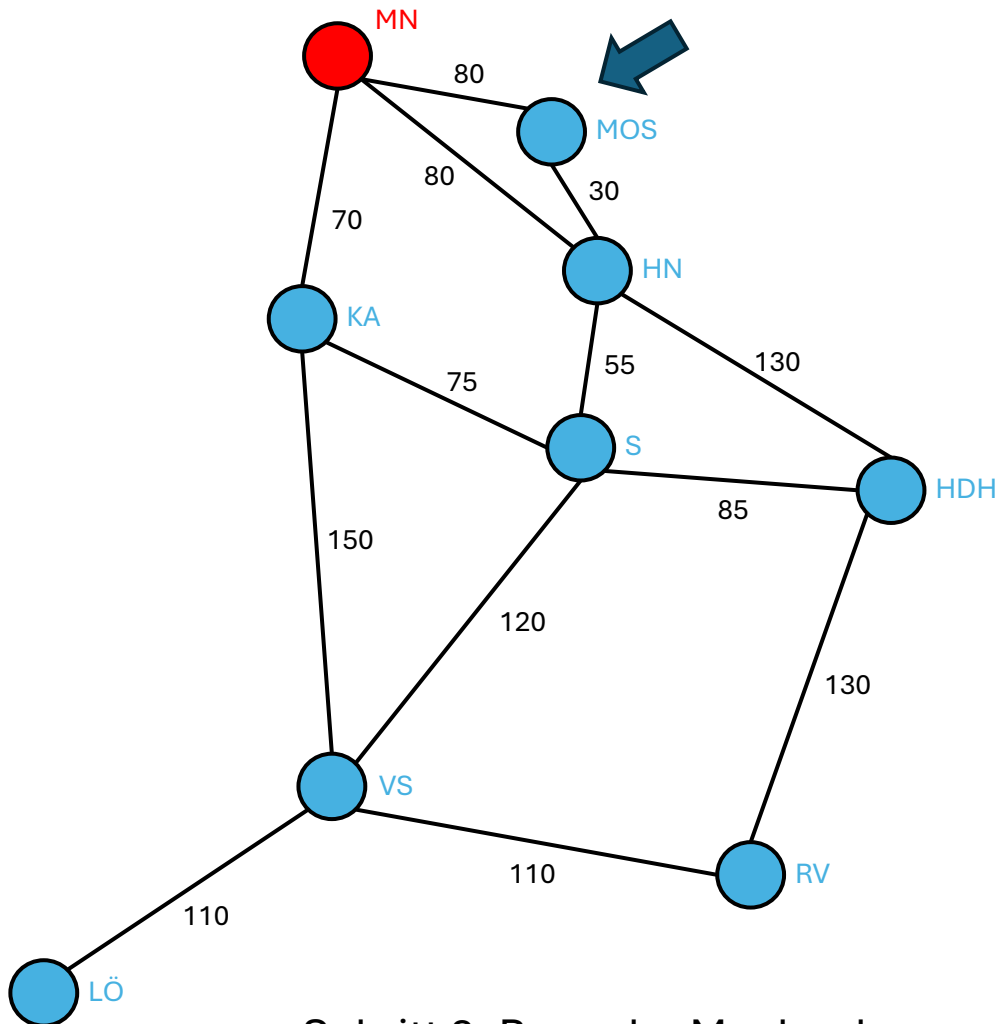
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 8: Besuche Mosbach

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	315	285	260	0	330	315	220	230	110
$\text{prev}[v]$	S	S	VS	-	KA	HN	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

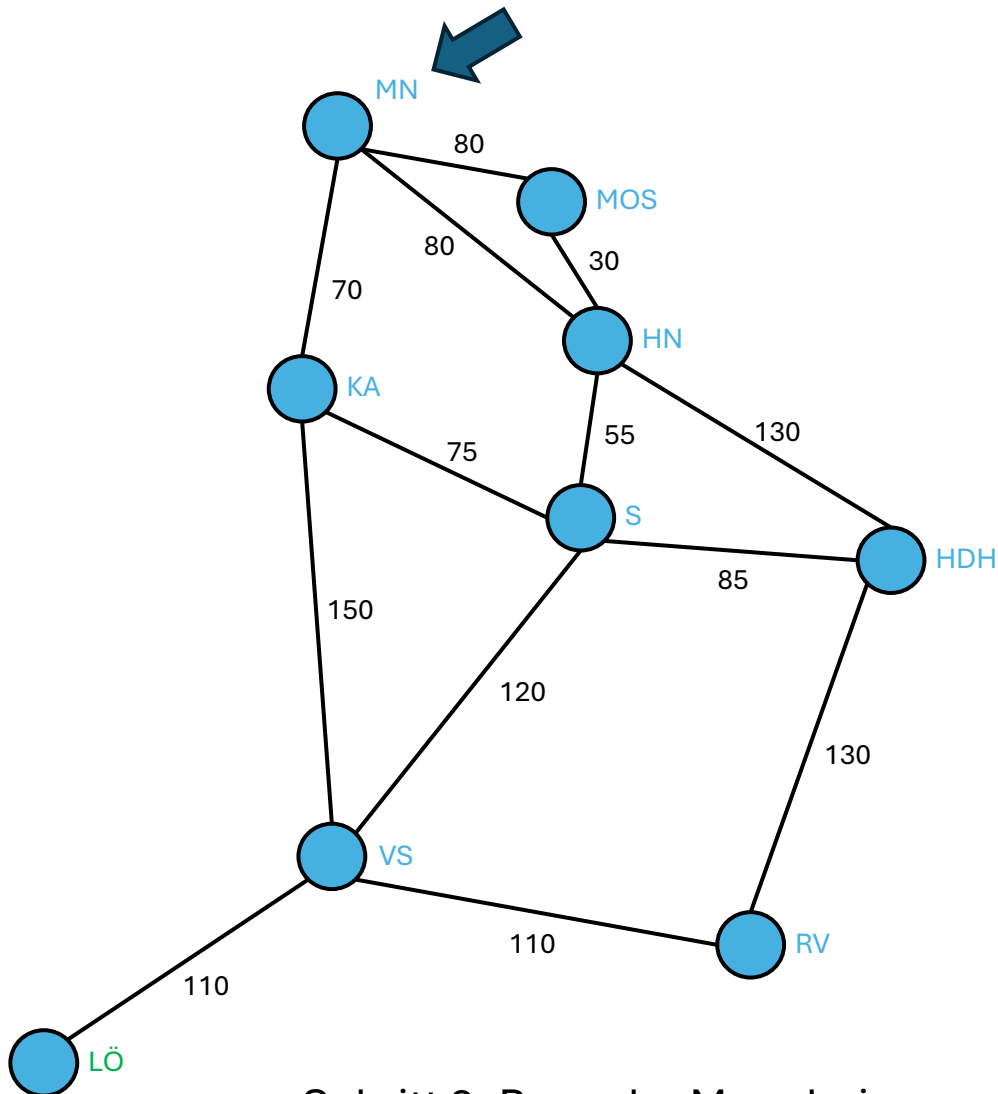
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Schritt 9: Besuche Mannheim

Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige



v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	315	285	260	0	330	315	220	230	110
$\text{prev}[v]$	S	S	VS	-	KA	HN	VS	VS	LÖ

while Q is not empty:

$u \leftarrow Q.\text{extractMin}()$ // entferne Knoten mit minim. Distanz aus Q

for each neighbor v of u : // Betrachte alle Nachbarn von u

if $\text{dist}[v] == \infty$: // falls bislang nie besucht

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{addWithPriority}(v, \text{dist}[v])$

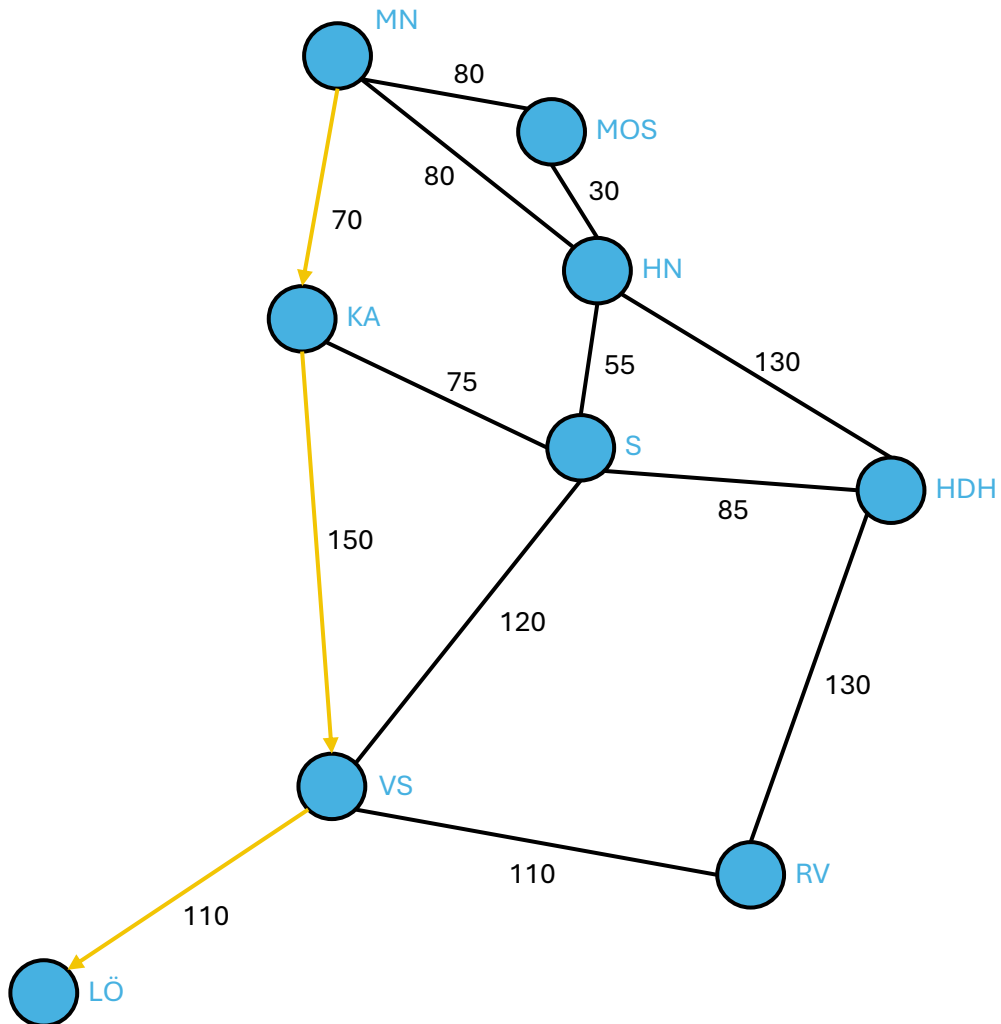
elseif $\text{dist}[u] + w(u, v) < \text{dist}[v]$: // falls neue kleinere Distanz

$\text{prev}[v] \leftarrow u$ // Setze u als Vorgänger

$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$ // Distanz zu v via u

$Q.\text{decreasePriority}(v, \text{dist}[v])$

Dijkstra Beispiel



Knotenarten:



Bereits besucht



Kandidat in Prioritätsliste Q



Sonstige

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
dist[v]	315	285	260	0	330	315	220	230	110
prev[v]	S	S	VS	-	KA	HN	VS	VS	LÖ



Kürzeste Wege durch **Rückverfolgen** der Vorgängerlinks

z.B. kürzeste Route von Lörrach nach Mannheim*:

Lörrach → Villingen-Schwenningen → Karlsruhe → Mannheim
(330km)

Dijkstra – Komplexität

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
dist[v]	315	285	260	0	330	315	220	230	110
prev[v]	S	S	VS	-	KA	HN	VS	VS	LÖ

- **Speicher**

- $O(|V|)$ + Speichern von Kanten

- **Zeit-/Rechenkomplexität**

- Dominiert durch Operationen auf Prioritätsliste Q
- Bei Verwendung eines Heaps sind
 - extractMin()
 - addWithPriority()
 - decreasePriority()
 in $O(\log(|V|))$ umsetzbar
- Schleife wird bis zu $|V|$ mal durchlaufen
- Aber: Jede Kante(nrichtung) wird über alle Schleifen hinweg nur 1x betrachtet
- Das ergibt $O(|E| * \log(|V|))$ *

* Ein Paper von 2025 führt eine neue Datenstruktur ein, die die Komplexität auf $O(|E| * \log^{2/3}(|V|))$ reduziert (Duan et al.), praktische Relevanz tbd

function Dijkstra($G(V, E, w)$, s)

```

// Initialisiere
Q = []
Q.addWithPriority(source, 0)
for each vertex v in V:
    dist[v] ← ∞
    prev[v] ← '-'

dist[LÖ] ← 0

// Iteration
while Q is not empty:
    u ← Q.extractMin() // entferne Knoten mit minim. Distanz aus Q

    for each neighbor v of u: // Betrachte alle Nachbarn von u
        if dist[v] == ∞: // falls bislang nie besucht
            prev[v] ← u // Setze u als Vorgänger
            dist[v] ← dist[u] + w(u, v) // Distanz zu v via u
            Q.addWithPriority(v, dist[v])
        elseif dist[u] + w(u, v) < dist[v]: // falls neue kleinere Distanz
            prev[v] ← u // Setze u als Vorgänger
            dist[v] ← dist[u] + w(u, v) // Distanz zu v via u
            Q.decreasePriority(v, dist[v])

// Rückgabe
return dist[], prev[]

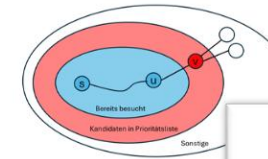
```

Dijkstra – Optimalität

- Dijkstra ist eine **gewichtete Form der Breitensuche** (Gewichte =1: Breitensuche)
- Dijkstra **garantiert den kürzesten Pfad** vom Startknoten zu allen anderen Knoten, wenn der Graph **nicht negative** Kantengewichte hat
 - Kann **induktiv bewiesen** werden
- Weiterhin:
 - Nachdem ein Knoten v besucht wurde, verbessert sich sein Distanzwert $\text{dist}[v]$ nicht weiter
 - Falls nur Distanz zu Knoten v gesucht: Abbruch möglich, sobald v besucht wurde
 - Aber...

Dijkstra – Optimalität

• Beweis per Induktion

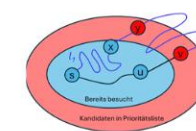


Behauptung:

Falls Knoten v als nächstes besucht wird, dann ist $\text{dist}[v]$ die Länge eines kürzesten Weges von s nach v , d.h. $\text{dist}[v] = \min \{w(p_{s,v}) \mid p_{s,v} \text{ ist Pfad von } s \text{ nach } v\}$

Induktion über die Anzahl der besuchten Knoten:

Dijkstra – Optimalität



Falls Knoten v als nächstes besucht wird, dann ist $\text{dist}[v]$ die Länge eines kürzesten Weges von s nach v , d.h. $\text{dist}[v] = \min \{w(p_{s,v}) \mid p_{s,v} \text{ ist Pfad von } s \text{ nach } v\}$

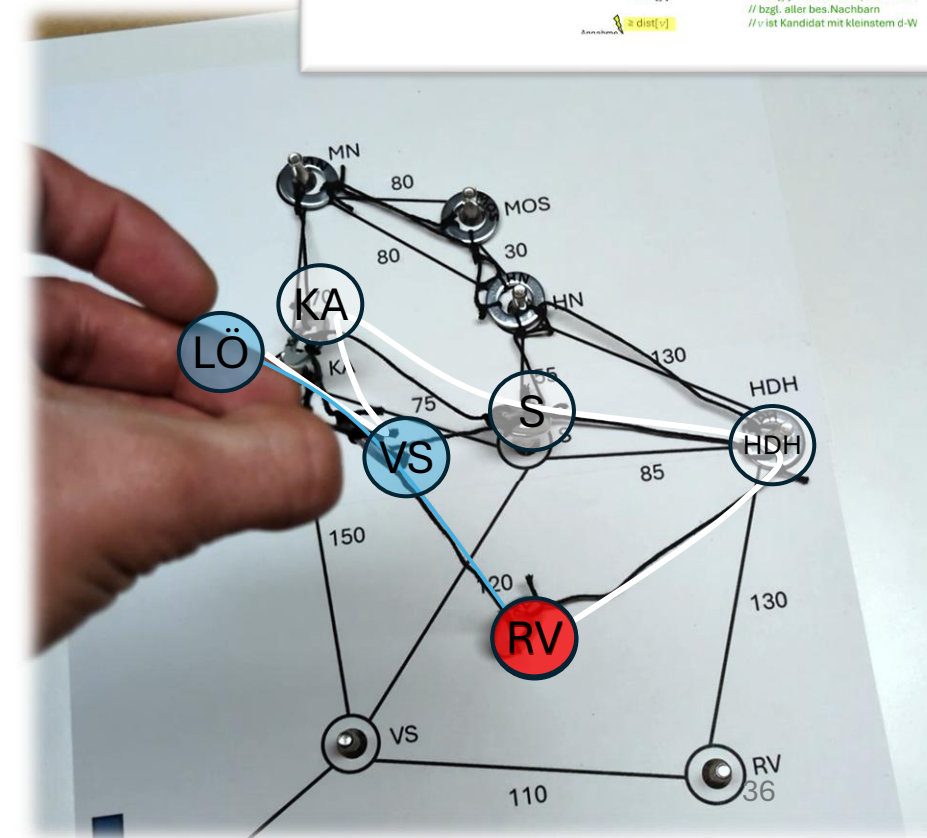
Induktionsschritt:

Nun müssen wir zeigen: Wenn die Behauptung für alle Schritte bis hierhin gilt (Induktionsannahme), so gilt sie auch für den nächsten Schritt, in dem v besucht wird

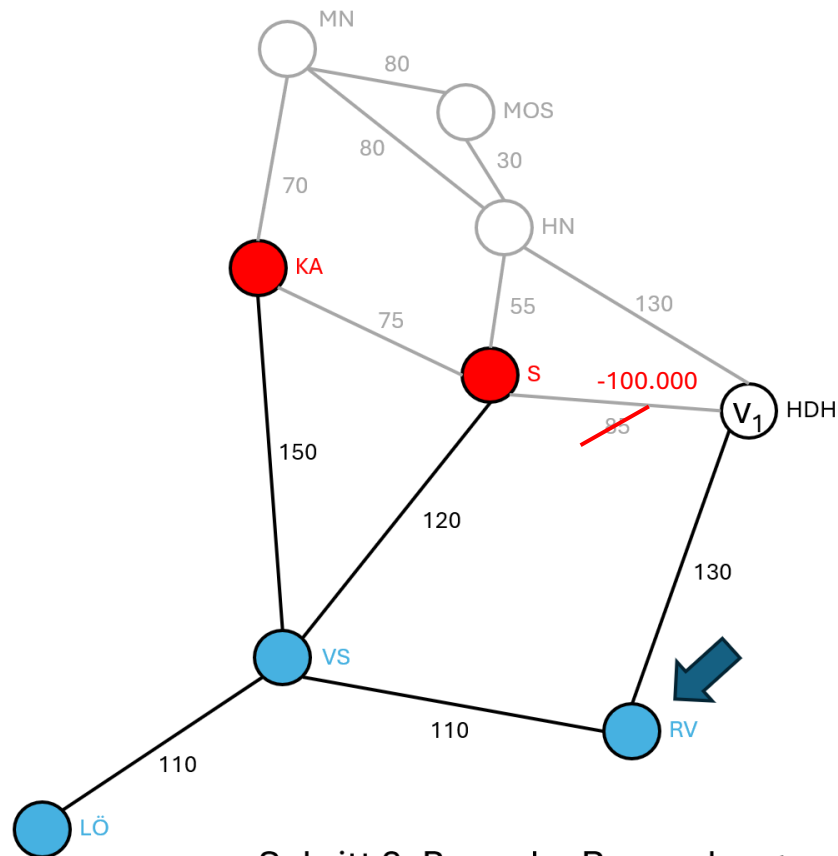
Beweis durch Widerspruch

- Annahme: Es gibt einen anderen kürzeren Weg $p_{s,v}$ mit Länge $< \text{dist}[v]$
- Weg p muss über einen Kandidaten gehen
- y sei der erste Kandidaten-Knoten im Weg p (ggf. $y=v$)
- Der Vorgänger auf diesem Weg sei $p \in v[y]=x$ und muss bereits besucht worden sein

$l = l_1 + w(x,y) + l_2$
 $\geq \min \{w(p_{s,x})\} + w(x,y) + 0 \quad // l_1 \geq \text{dist}[s,x] \text{ und } l_2 \geq 0$
 $= \text{dist}[x] + w(x,y) \quad // \text{Ind. ann. für besuchten Knoten}$
 $\geq \text{dist}[y] \quad // \text{dist}[y] \text{ ist kürzeste Option nach } y$
 $\geq \text{dist}[v] \quad // \text{bzgl. aller bes. Nachbarn}$
 $\geq \text{dist}[v] \quad // v \text{ ist Kandidat mit kleinstem d-W}$



Dijkstra scheitert an negativen Gewichten



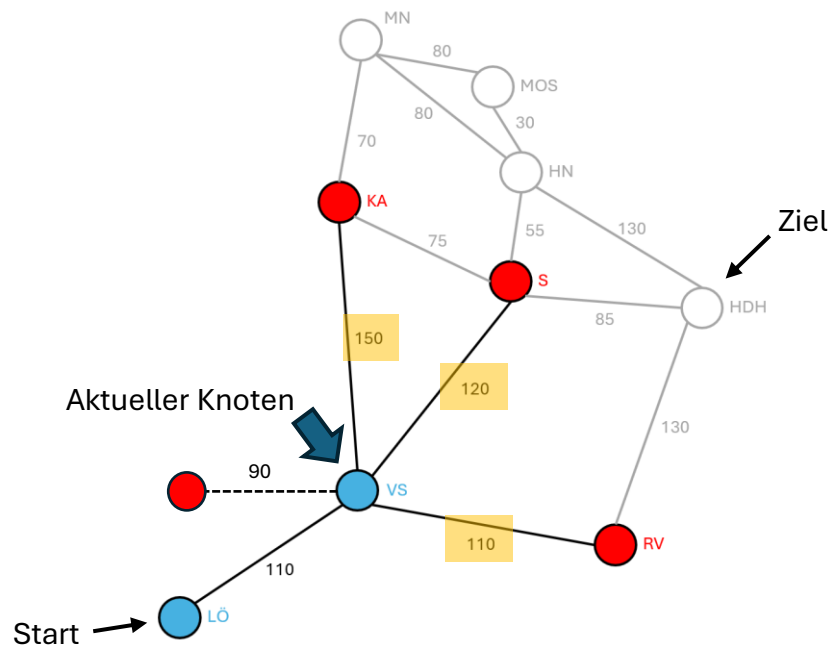
Schritt 3: Besuche Ravensburg

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
$\text{dist}[v]$	350	∞	260	0	∞	∞	220	230	110
$\text{prev}[v]$	RV	-	VS	-	-	-	VS	VS	LÖ

- Optimum wird ggf. **verpasst**, wenn es negative Gewichte gibt
- Einmal besucht, wird Distanz von Knoten **nicht mehr angepasst**

Dijkstra ist „gierig“

- **Greedy** (gieriger) Algorithmus – manchmal ineffizient
- *Bsp.:* Kürzester Weg von LÖ nach HDH:



- Trifft **lokal beste** Entscheidung
- Kann ineffizient sein, wenn wir an einem bestimmten Ziel interessiert sind

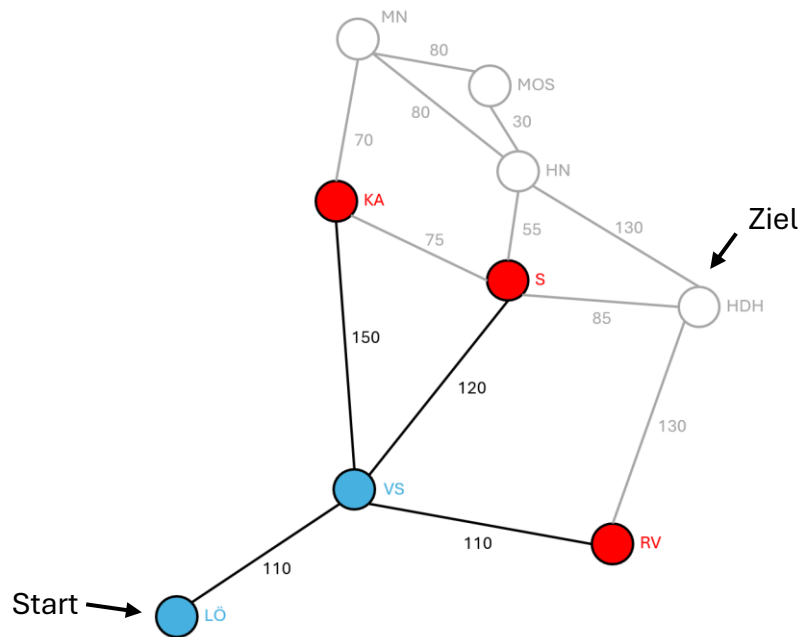
Suchverfahren in der KI

*Teil 3: Heuristische Suchverfahren, A**



Idee des A* Algorithmus

- *Einschränkung*: Berechnet nur den kürzesten Weg zwischen **zwei** Knoten
- *Neu*: **Suchheuristik** h , um den weiteren Weg bis zum Ziel **abzuschätzen**



Gesucht: Optimistische Abschätzung für unbekannte echte Distanz

Neue Prio ergibt sich aus:

$$\text{dist}[u] + w(u, v)$$

$$\text{dist}[u] + w(u, v) + h(v, t)$$

Heuristik: Luftlinie (unterschätzt immer!)

	HN	KA	LÖ	MN	MOS	RV	S	VS
HDH	85	135	220	150	105	105	70	140

$$\begin{aligned} \text{prio}[KA] &= 110 + 150 + 135 = 395 \\ \text{prio}[RV] &= 110 + 110 + 105 = 325 \\ \text{prio}[S] &= 110 + 120 + 70 = 300 \end{aligned}$$

v	HDH	HN	KA	LÖ	MN	MOS	RV	S	VS
prio[v]	∞	∞	395	0	∞	∞	325	300	110
prev[v]	-	-	VS	-	-	-	VS	VS	LÖ

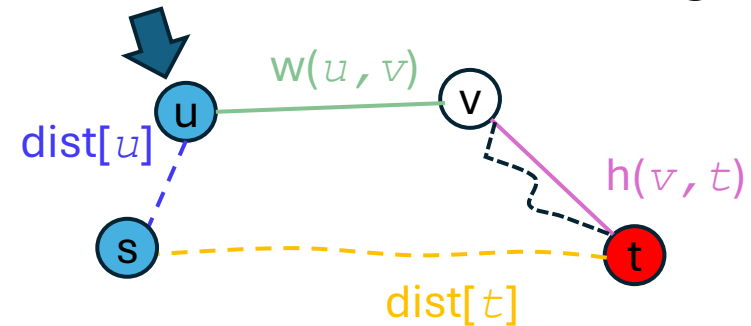
```
function Dijkstra(G(V, E, w), s)
    // Initialisiere
    Q = [] // Prioritätsliste
    Q.addWithPriority(source, 0) // Füge Startknoten zu Kandidatenliste hinzu
    for each vertex v in V:
        dist[v] ← ∞ // Distanz vom Startknoten
        prev[v] ← '-' // kein Vorgängerknoten
    dist[LÖ] ← 0 // Distanz zum Startknoten

    // Iteration
    while Q is not empty:
        u ← Q.extractMin() // entferne Knoten mit minim. Distanz aus Q
        for each neighbor v of u: // Betrachte alle Nachbarn von u
            if dist[v] == ∞: // falls bislang nie besucht
                prev[v] ← u // Setze u als Vorgänger
                dist[v] ← dist[u] + w(u, v) // Distanz zu v via u
                Q.addWithPriority(v, dist[v])
            elseif dist[u] + w(u, v) < dist[v]: // falls neue kleinere Distanz
                prev[v] ← u // Setze u als Vorgänger
                dist[v] ← dist[u] + w(u, v) // Distanz zu v via u
                Q.decreasePriority(v, dist[v])

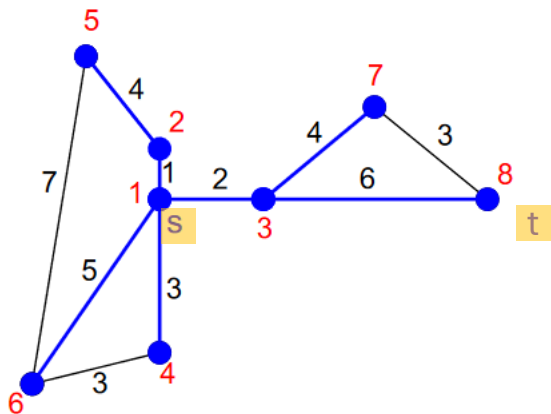
    // Rückgabe
    return dist[], prev[]
```

Eigenschaften des A* Algorithmus

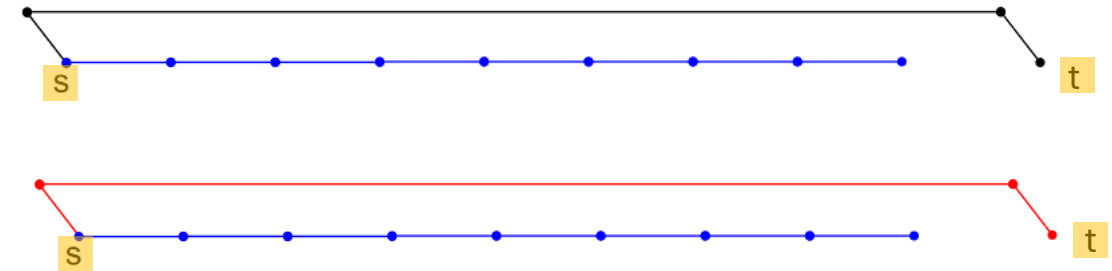
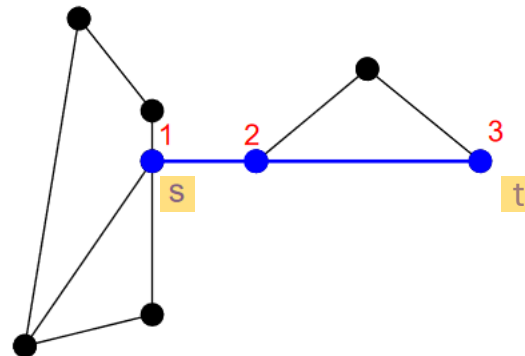
- ...erlaubt ein **zielgerichteteres Suchen**
- Funktioniert nur, wenn die **Heuristik die echte Entfernung *niemals überschätzt***:
Algorithmus bricht ab, wenn kein Knoten u mit Nachbarknoten v mehr in Q , für den gilt
$$\text{dist}[u] + w(u, v) + h(v, t) < \text{dist}[t]$$
- Für $h=0$ erhält man wieder **Dijkstra**



Dijkstra

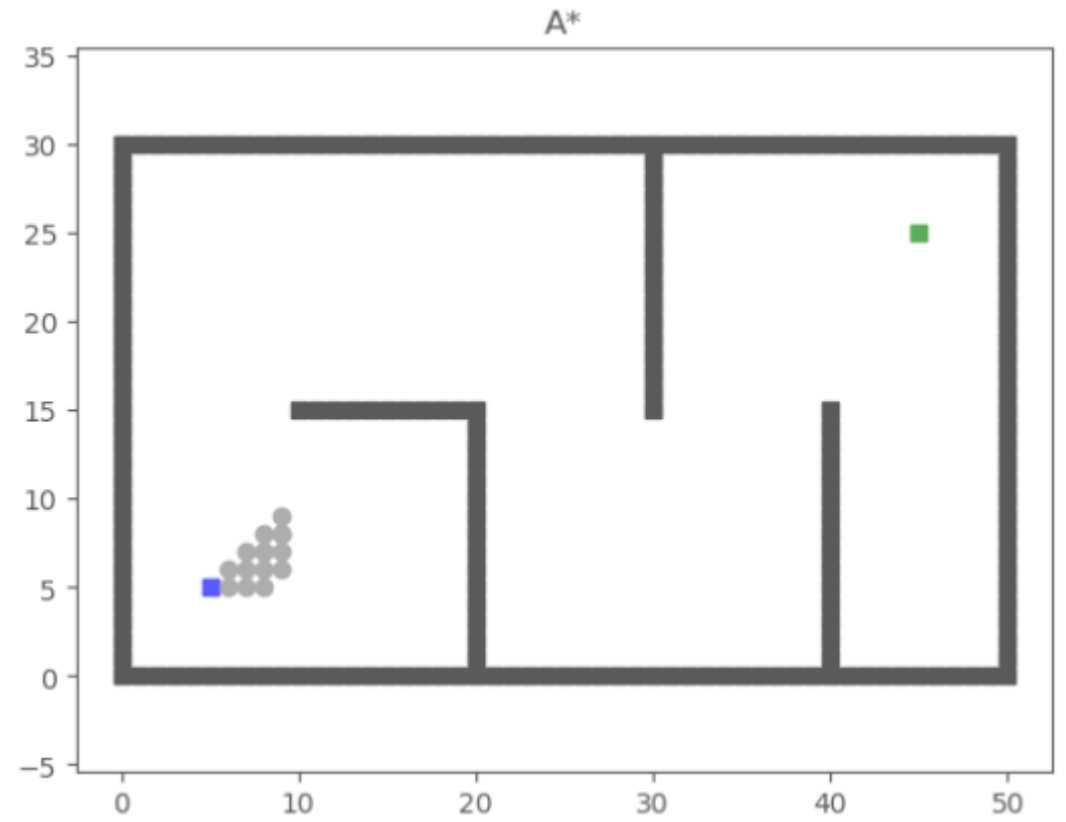
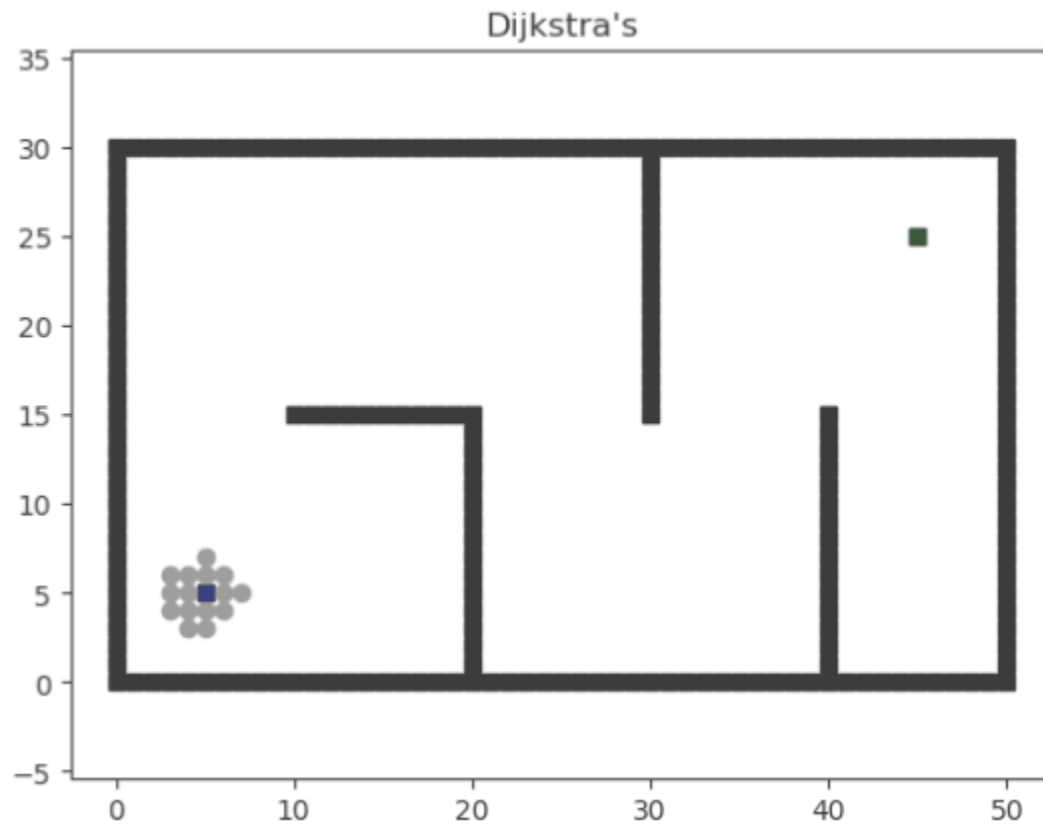


A*



Quelle: https://www-home.htwg-konstanz.de/~bittel/ain_alda/Vorlesung/09_Graphen_KuerzesteWege.pdf

Dijkstra vs. A*



Source: <https://github.com/zhm-real/PathPlanning>

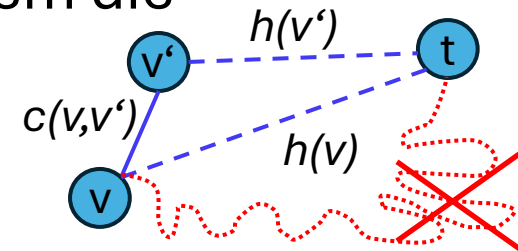
Damit es funktioniert: Heuristik muss „zulässig“ sein

- Eine **Heuristik** heißt „**zulässig**“ („**admissible**“), wenn sie *niemals* die echten Kosten *überschätzt*, d.h. $h(v) \leq h^*(v) \forall v$, wobei $h^*(v)$ die wahren minimalen Kosten von v bis zum Ziel sind. Da Heuristiken typischerweise ≥ 0 sind, folgt, dass $h(\text{Ziel}) = 0$
- Manchmal findet man in der Literatur synonym die Bezeichnung „**optimistisch**“
- Eine Heuristik heißt „**monoton**“ oder „**konsistent**“ wenn außerdem die Dreiecksungleichung gilt, d.h.

$$h(v) \leq c(v, v') + h(v') \quad \forall \text{ Kanten } (v, v'),$$

wobei $c(v, v')$ die tatsächlichen Kosten der Kante (v, v') sind

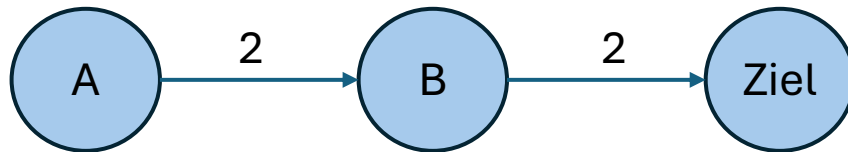
- d.h. der Heuristikwert ist nicht größer als der Weg über einen direkten Nachbarn plus dessen Heuristik
- Anders gesagt: wenn man von v nach v' geht, darf die Heuristik nicht stärker abfallen als die tatsächlichen Kosten dorthin (z.B.: 100km Richtung Ziel gefahren, Heuristik um 200km gesenkt)
- *Bsp.:* Euklidische Distanz in einem Straßennetz: die Luftlinie verletzt nie die Dreiecksungleichung



Warum ist eine konsistente Heuristik eine gute Idee?

- Eine **konsistente Heuristik ist immer auch zulässig** (denn: die Ungleichung gilt für jede Kante)
- Wegen $h(v) \leq c(v, v') + h(v')$ folgt, dass die Summe $f(v)$ der tatsächlichen Kosten bis zum Knoten v plus dessen Heuristik $h(v)$ stets kleiner oder gleich der tatsächlichen Kosten bis zum Knoten v' plus dessen Heuristik $h(v')$ ist, d.h.
 - entlang eines optimalen Pfades werden die f -Werte nie kleiner
 - es kann also nicht passieren, dass ein später gefundener Pfad zu einem Knoten billiger ist als der erste
 - Knoten müssen folglich nur ein mal und nicht erneut betrachtet werden

Umgekehrt gilt das nicht immer...



Tatsächlich optimale Kosten:

- $h^*(Ziel) = 0$
- $h^*(B) = 2$
- $h^*(A) = 4$

Wir definieren die Heuristik wie folgt:

- $h(Ziel) = 0$
- $h(B) = 1$
- $h(A) = 4$

h ist **zulässig**, da:

- $h(Ziel) = h^*(Ziel) = 0$
- $h(B) = 1 < h^*(B) = 2$
- $h(A) = h^*(A) = 4$

...aber **nicht konsistent**, denn:

- $h(A) = 4 \not\leq c(A, B) + h(B) = 2 + 1 = 3$

Aufgabe

- Gegeben:
 - Ungerichteter Graph mit Startknoten s und Zielknoten t und positiven Kantengewichten
 - $h^*(v)$: optimale, d.h. kürzeste, Entfernung von einem Knoten v zum Zielknoten t
 - h : zulässige Heuristik
 - A* Suche mit einer Heuristik h' führt zu einer Lösung mit Kosten C
- Welche Aussagen treffen zu? Wählen Sie die jeweils am besten passendste!
 1. Falls $h'(v) = \frac{1}{2}h(v)$ für alle Knoten v :
[] $C = h^*(v)$ [] $C > h^*(v)$ [] $C \geq h^*(v)$
 2. Falls $h'(v) = \frac{h(v)+h^*(v)}{2}$ für alle Knoten v :
[] $C = h^*(v)$ [] $C > h^*(v)$ [] $C \geq h^*(v)$
 3. Falls $h'(v) = h(v) + h^*(v)$ für alle Knoten v :
[] $C = h^*(v)$ [] $C > h^*(v)$ [] $C \geq h^*(v)$

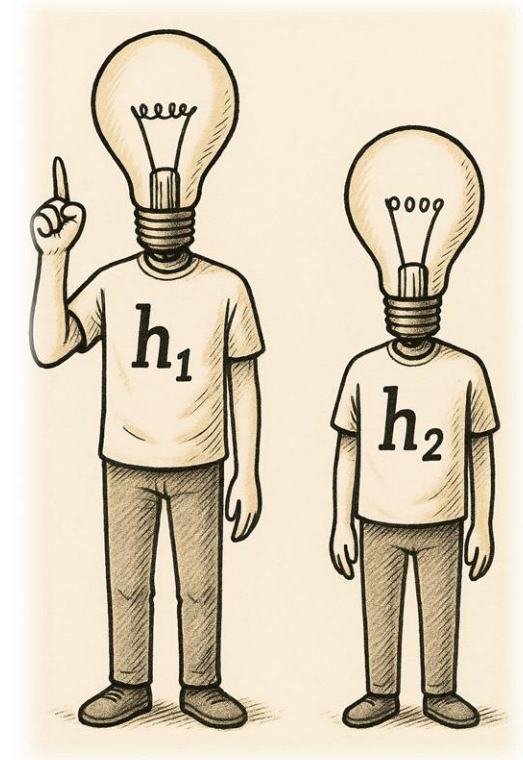
Dominanz

- Eine Heuristik h_1 **dominiert** eine andere Heuristik h_2 wenn sie mindestens genauso informativ ist, d.h.

$$h_1(v) \geq h_2(v) \text{ für alle Knoten } v$$

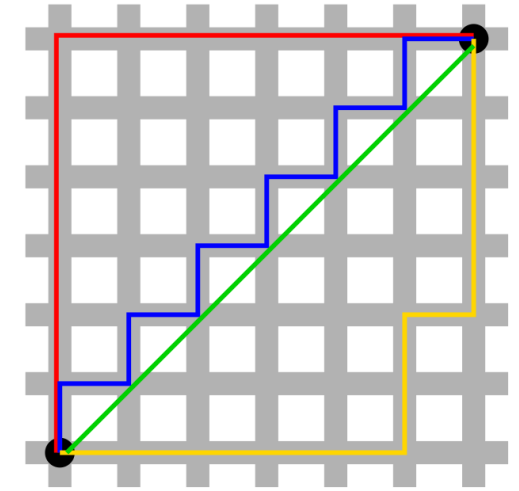
und beide zulässig sind

- Das bedeutet: h_1 liefert stets höhere Werte als h_2 (oder gleich große), überschätzt die Entfernung aber trotzdem nie (da h_1 zulässig ist)
- Die dominierende Heuristik h_1 ist in der Regel **zu bevorzugen**, denn h_1 schätzt die verbleibenden Restkosten besser ab, ist also „informierter“!
- Beispiel:
 - h_1 : Luftlinie von v zum Ziel
 - h_2 : Gleich 0 für alle Knoten ($\rightarrow$ uninformierte Suche)



Beispiel: 8-Puzzle

- Gegeben sind zwei Heuristiken
 - $h_1(v) = \text{Anzahl falsch platzierter Kacheln}$
 - $h_2(v) = \text{Summe der Manhattan – Distanzen jeder Kachel von ihrer Zielposition}$



Manhattan-Distanz: rot/blau/gelb vs.
Euklidische Distanz (grün)

- Aufgaben
 1. Welche dieser Heuristiken sind zulässig. Begründen Sie!
 2. Welche der beiden ist dominanter?
 3. Wäre die Summe beider Heuristiken zulässig?

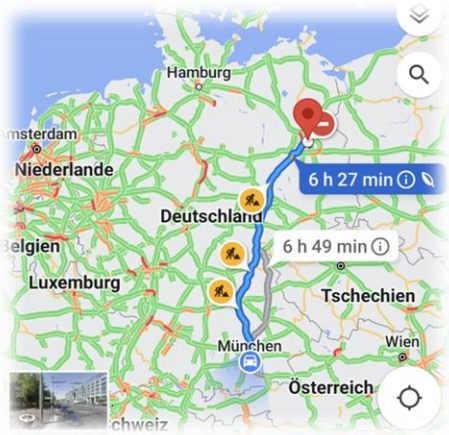
Initial State			Goal State		
1	2	3	2	8	1
8		4		4	3
7	6	5	7	6	5

Reflexion: Suche

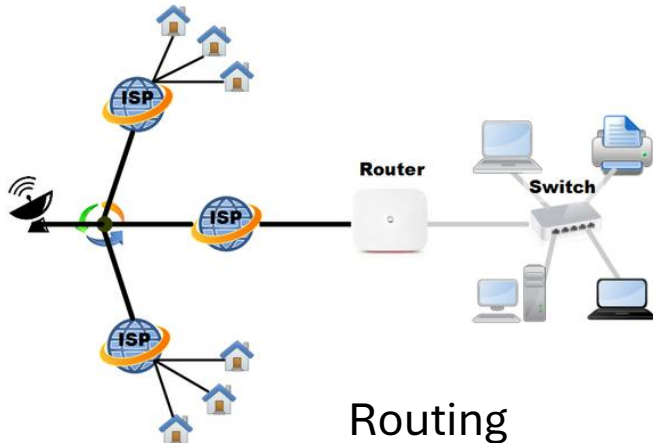
- Was ist der Unterschied zwischen informierter und uninformatierter Suche?
- Was ist eine Heuristik?
- Wann ist eine Heuristik zulässig/konsistent/dominant?

Reflektieren Sie kurz die Fragen und besprechen Sie die Antworten mit Mitstudierenden.

Anwendungsbeispiele



Navigation



Routing

Michael Hanselmann

DER SPIEGEL

10.05.2012

Drucken | Senden | Feedback | Markieren

Netzwerk-Studie

Was Facebook über Nicht-Mitglieder weiß

Von Konrad Lischka



Facebook-Chef vor Netzwerk-Graph: Mark Zuckerbergs Firma kennt auch Nicht-Mitglieder

Wer kennt wen? Wie viel kann Facebook über Menschen wissen, die gar nicht bei Facebook sind? Deutsche Forscher haben mit einer Simulation gezeigt: Mit den Daten des sozialen Netzwerks lässt sich vorhersagen, ob Menschen einander kennen, die dort gar kein Profil haben.

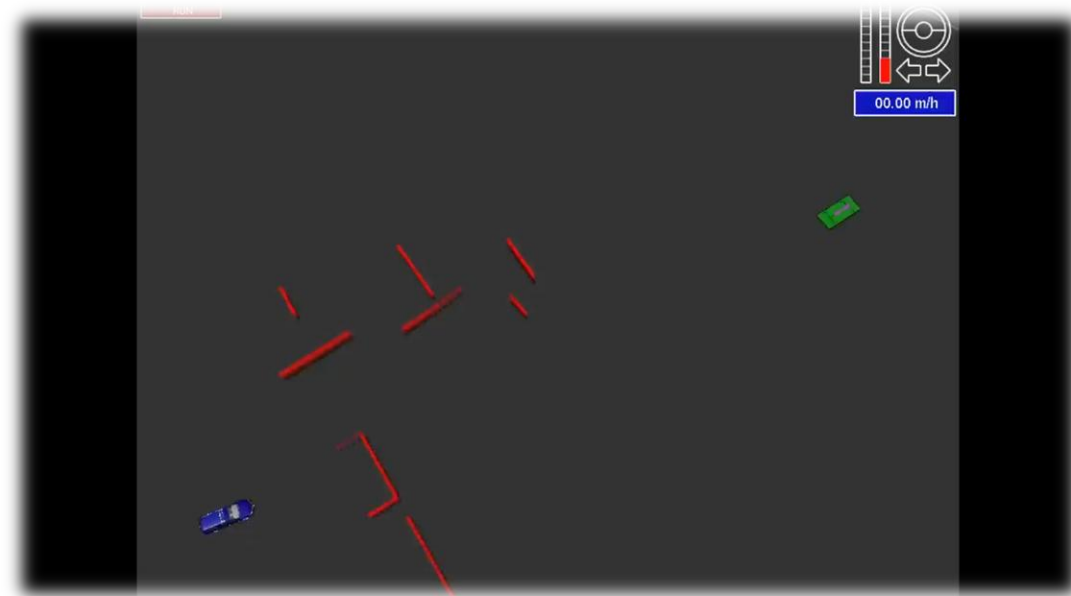
[Agnes Horvath, M Hanselmann et al. 2012](#)

Soziale Netzwerke

Künstliche Intelligenz



Gaming: Pfadfinden in Unity



Automatisiertes Fahren ⁵⁰
Source: Udacity

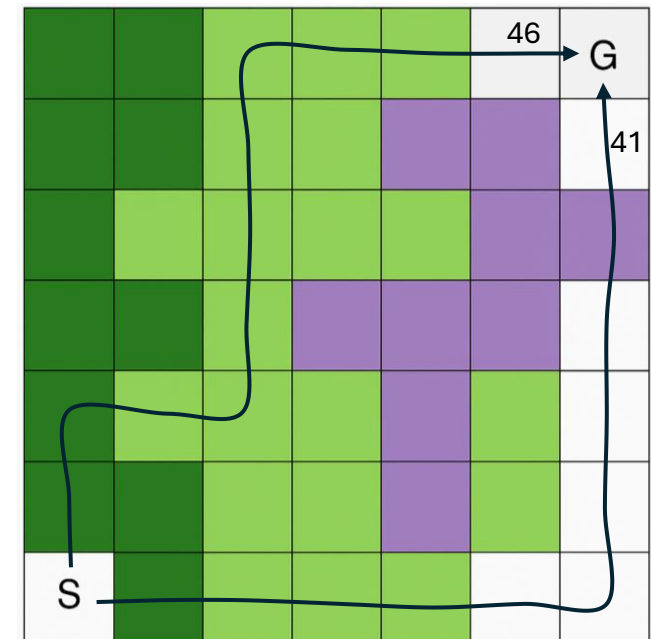
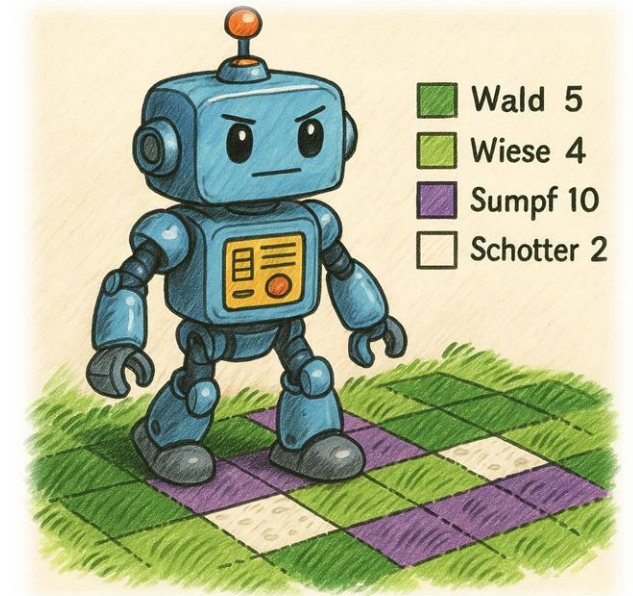
Beispiel: Roboternavigation

- **Problemstellung:**

- Roboter soll auf kostengünstigstem Weg von Start zu Ziel navigieren in Grid
- Unterschiedliche Geländekosten je Zelle
 - Schotter (weiss): 2
 - Wiese (hellgrün): 4
 - Wald (dunkelgrün): 5
 - Sumpf (lila): 10

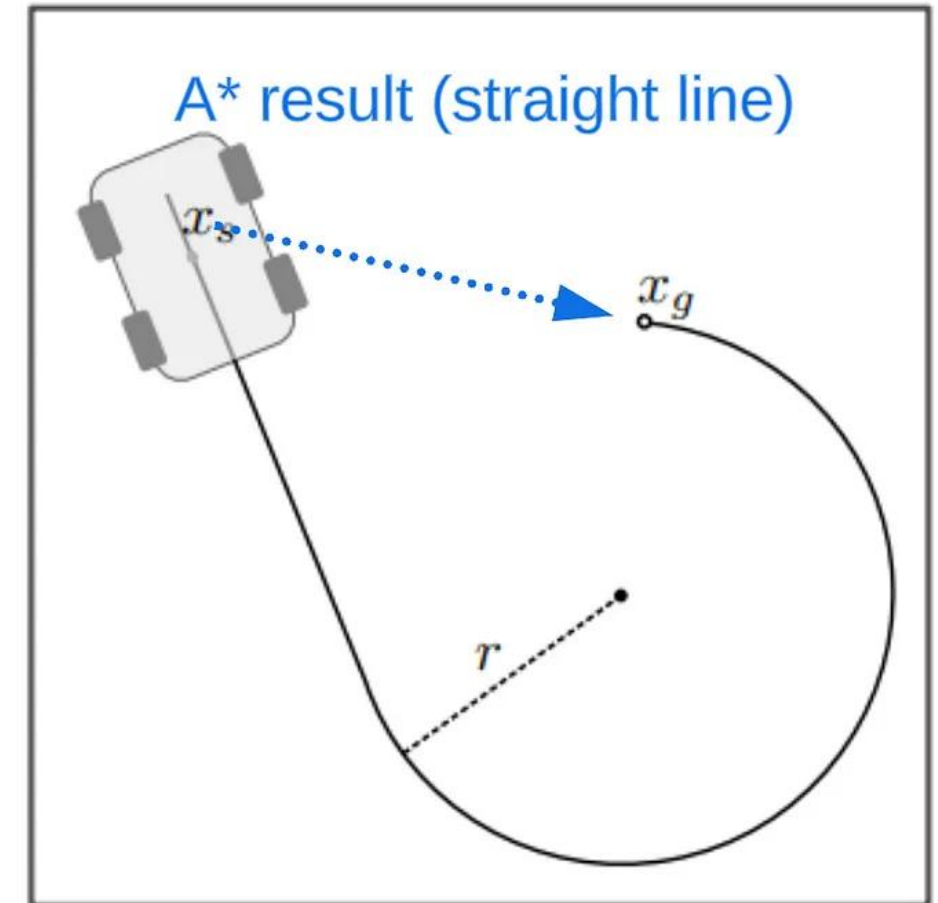
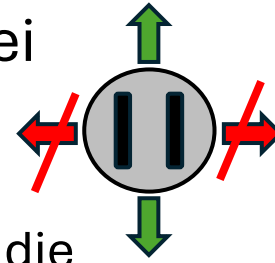
- **Aufgaben**

1. Geben Sie zwei zulässige, nicht-triviale, Heuristiken an!
2. Überlegen Sie, ob/welche Heuristik dominiert!



Funktioniert in der Grid-World... aber: was ist mit der Realität?

- A* hat signifikante Einschränkungen bei Anwendung auf **nicht-holonome* Systeme** (z.B. Auto)
 - Ein nicht-holonomes Constraint schränkt die zulässigen Beschleunigungen weiter ein
 - z.B., dass sich ein Fahrzeug / Roboter nur in manche Richtungen (vorwärts, rückwärts), nicht aber in andere (seitwärts) bewegen kann
- *Beispiel:* A* liefert den blauen Pfad, der in einem Grid (global) optimal ist
 - *Aber:* diese Lösung ist bei der gegebenen Orientierung des Fahrzeugs nicht (exakt) fahrbar
 - Folglich ist die Standard A* Lösung in der Praxis nicht besonders hilfreich



Source: <https://kth.diva-portal.org/smash/get/diva2:1057261/FULLTEXT01.pdf>

*Holonom: Constraints hängen nur von Positionen und ggf. Zeit ab

Hybrid A*

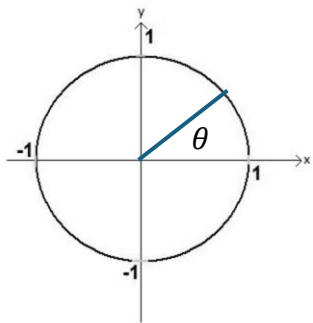
Link: <https://journals.sagepub.com/doi/abs/10.1177/0278364909359210> (1000+ Zitate)

- 2010 vorgestellt von Dmitri Dolgov (Waymo co-CEO)
- Überzeugende Performance bei der DARPA Challenge
- In den Folgejahren und bis heute häufig verwendeter Algorithmus in der Pfadplanung im Kontext assistiertes und automatisiertes Fahren und v.a. Parken (Niedriggeschwindigkeitsbereich)
- Python Code z.B. hier verfügbar:
<https://github.com/zhm-real/MotionPlanning>

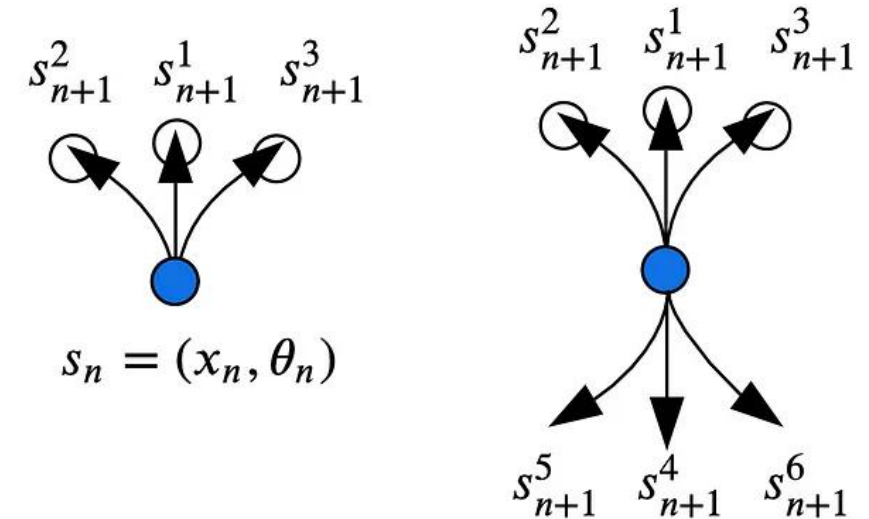


Hybrid A*: Kinematisches Modell

- **Standard A***: Finden eines kürzesten Pfades von einer Position x_0 im $\mathbb{R}^2$ (verallgemeinert: $\mathbb{R}^3$) zu einer Zielposition x_g
- **Hybrid A***: Finden einer Sequenz von fahrbaren Posen (x_i, θ_i) in $SE(2) = \mathbb{R}^2 \times S^1$
 - $SE(2)$: “Special Euclidean Group”
 - $\mathbb{R}^2$: Die 2D Position
 - S^1 : Winkel / Orientierung / Ausrichtung θ , also ein Punkt auf dem Einheitskreis



Im Paper: Fahrzeug-Modell



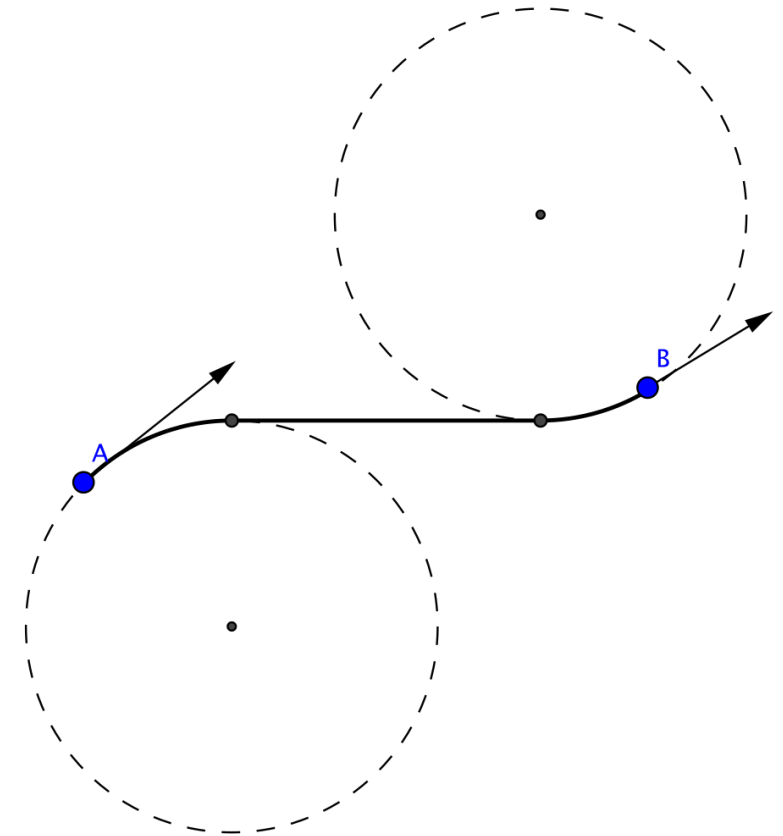
Bei der Expansion zu Schritt $n + 1$ werden nur **3 Bewegungsprimitive** berücksichtigt:

- Geradeaus / keine Lenkung
- Maximale Krümmung / Lenkeinschlag nach links
- Maximale Krümmung / Lenkeinschlag nach rechts

...analog nach vorne und nach hinten

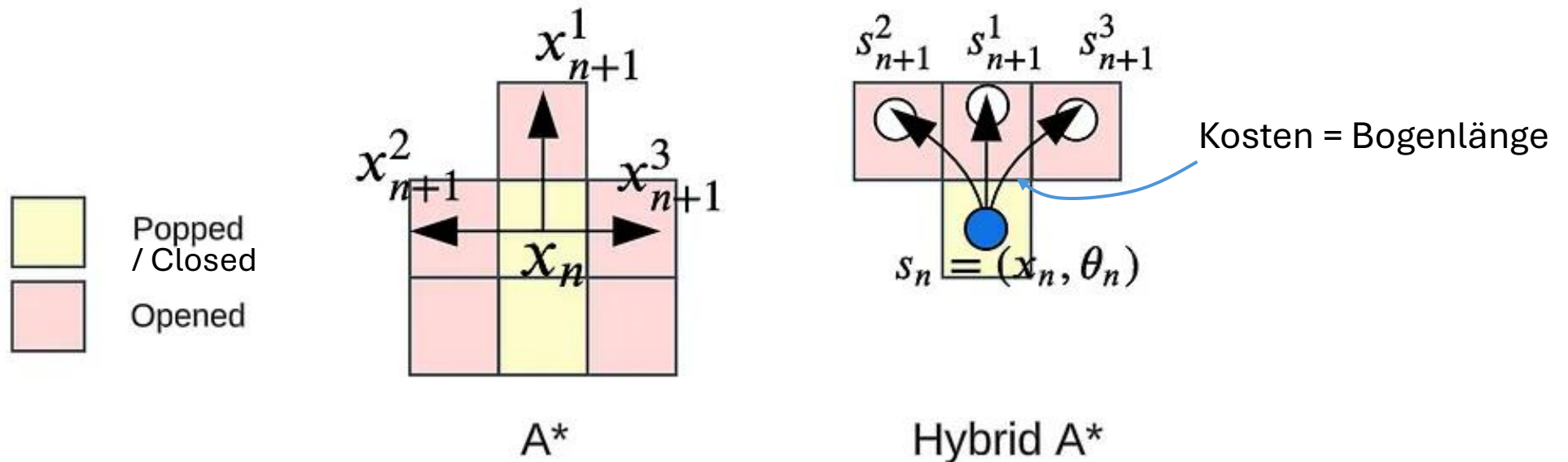
Hybrid A*: Kinematisches Modell

- Gegeben zwei Posen A und B , ist der kürzeste Pfad, der nur geradeaus, links oder rechts nutzt, der sogenannte **Dubins-Pfad**, dessen Länge analytisch berechnet werden kann
 - Unter den Annahmen:
 - Das Fahrzeug bewegt sich vorwärts mit konstanter Geschwindigkeit vorwärts
 - Es besitzt einen minimalen Wenderadius / eine maximale Krümmung
 - Der Lenkwinkel ist zwischen diesen Extremen beliebig wählbar
- gilt: *Der kürzeste Weg zwischen zwei beliebigen Posen in $SE(2)$ ist immer eine Abfolge von höchstens **drei Segmenten** mit maximaler Krümmung (L/R) oder null Krümmung (G)*
- Daher gibt es nur 6 mögliche Sequenzen für den optimalen Pfad: RGR , LGL , RGL , LGR , RGR , LRL/RLR ; man prüft alle und wählt den kürzesten (*Hinweis*: nur G ist darin enthalten)
 - Die **Dubin Distanz** ist also eine zulässige Heuristik und fahrbar-optimal



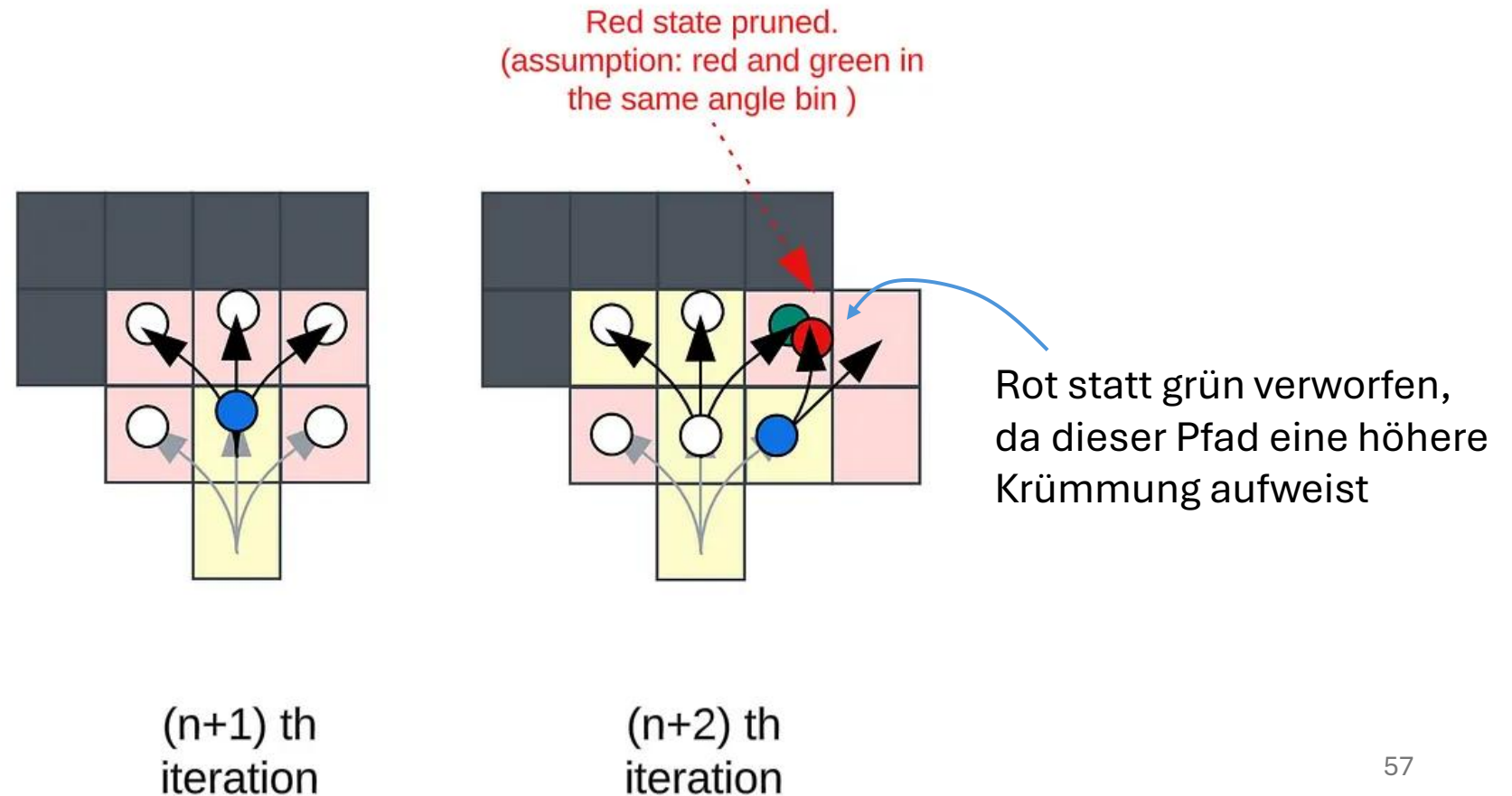
Hybrid A*: Expansion

- Beim **Standard A*** bewegen wir uns entlang des Rasters von Knoten zu Nachbarknoten, die Zustände entsprechen den Grid-Zentren
- Beim **Hybrid A*** werden die Nachbarzellen gemäß der zu Grunde liegenden Kinematic in die open Liste aufgenommen, außerdem wird nur im Ausnahmefall ein Gridzentrum erreicht



Hybrid A*: Expansion

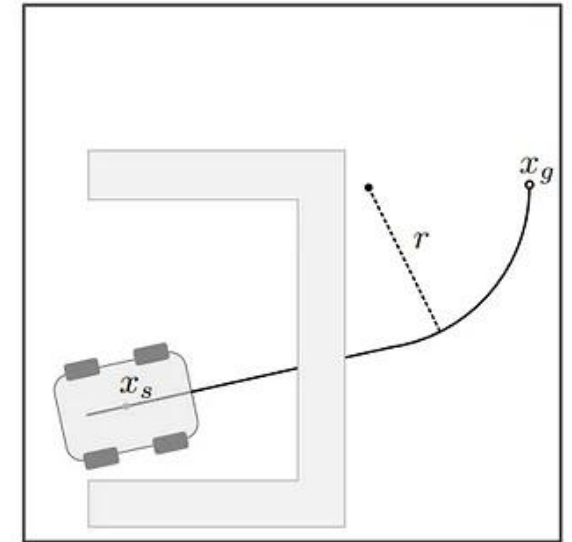
- Falls zwei Zustände **dieselbe Gridzelle** und eine **ähnliche Orientierung** teilen (typischerweise werden die Winkel diskretisiert, z.B. in 16 Bins mit je $22,5^\circ$), so „**prunt**“ (verwirft) man einen der beiden, um den Suchraum zu beschneiden



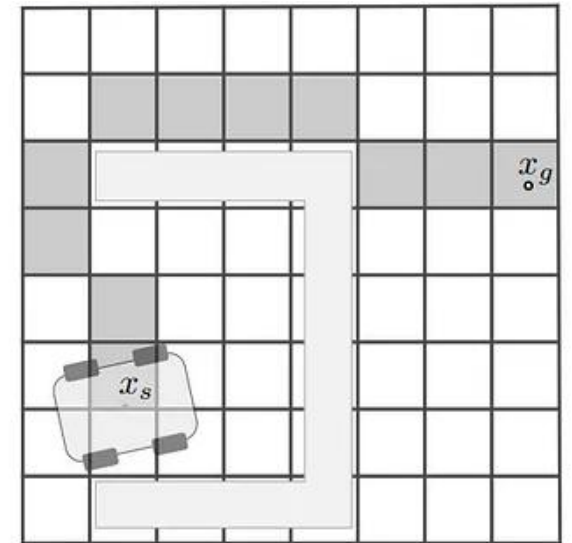
Hybrid A*: Heuristik

Der beste nächste Zustand wird nun, gewählt, indem, wie üblich, die Summe aus bisherigen Kosten und Heuristik bis zum Ziel betrachtet und die kleinste Option gewählt wird:

- *Aber:* Die vorgestellte Heuristik (Dubins-Distanz) kann **wenig informativ** sein, da Hindernisse nicht berücksichtigt werden
- Sollen Hindernisse berücksichtigt werden, wird die Fahrbarkeit typischerweise ignoriert und rein auf den Positionen gearbeitet
- Beides gleichzeitig zu betrachten, wäre viel zu rechenintensiv
- *Idee:* Wähle das **Maximum aus beiden Heuristiken** aus (fahrdynamische Dubins Heuristik, Hindernis-Heuristik)
 - **zulässig**, wenn beide einzeln zulässig
 - **dominanter** (größer) als jede Einzelheuristik
 - trotzdem kann $h(v)$ **weit unter** den echten Kosten $h^*(v)$ liegen



(c) The constrained heuristic heavily underestimating the cost of the path, due to ignoring obstacles.



(d) The unconstrained heuristic accounting for obstacles and dead ends.

Hybrid A*: Erweiterungen

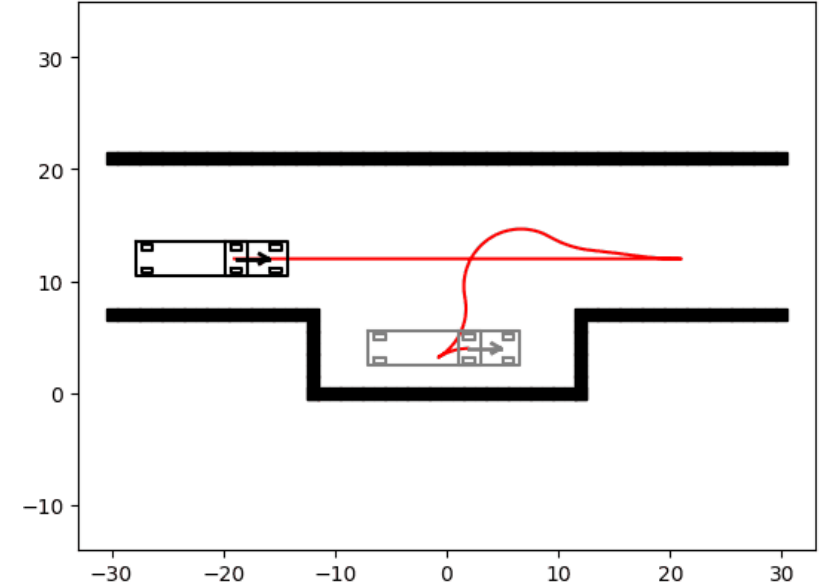
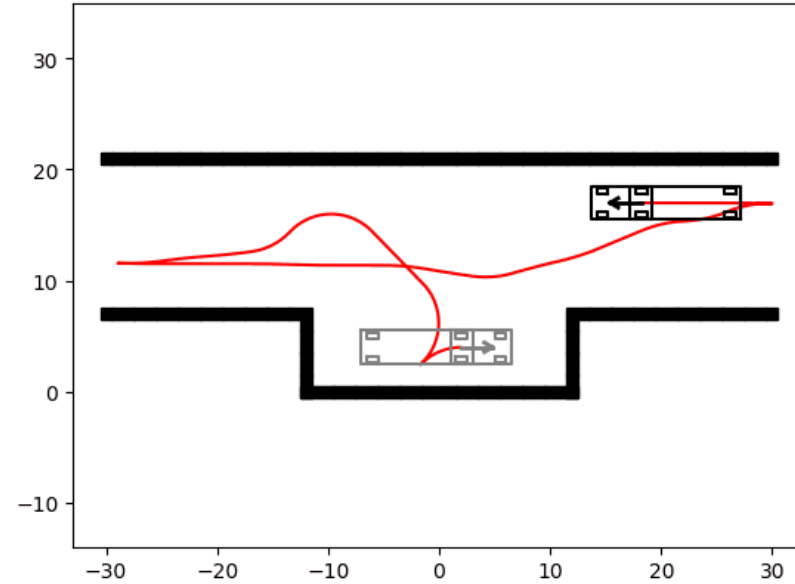
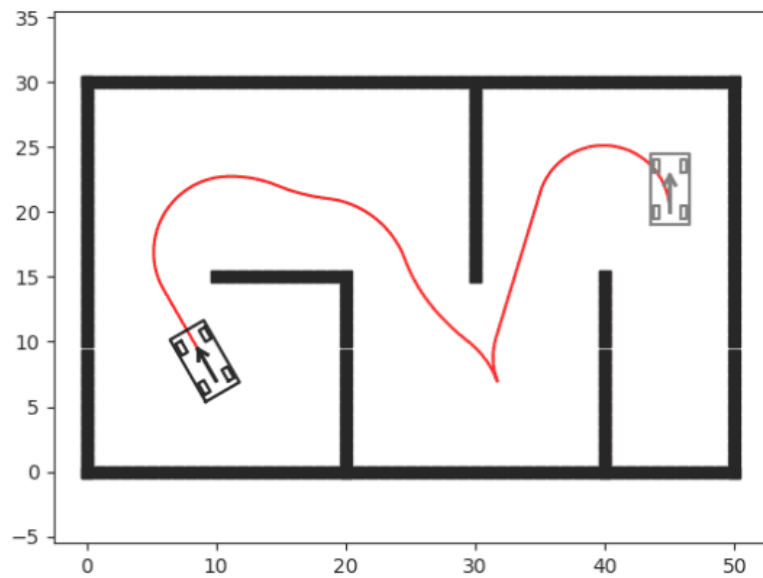
In Regionen mit wenig Hindernissen lohnt sich die „**Shot-to-Goal**“
Erweiterung:

- Von der aktuellen Pose aus wird versucht, das Ziel direkt mit einem Dubins-Pfad zu erreichen
- Wenn dies kollisionsfrei gelingt, kann die Suche beenden werden und wir haben eine optimale Lösung gefunden

Weiterhin gibt es Varianten, in denen (Hybrid) A* mit **Machine Learning** kombiniert wird, etwa

- Zum Lernen der Heuristik (aber: Zulässigkeit nicht garantiert!)
- Zur Prädiktion anderer Verkehrsteilnehmer / dynamischer Hindernisse

Hybrid A*: Beispiele



Hybrid A*: Vor- und Nachteile

Vorteile:

- + robust, deterministisch
- + Einfache Implementierung, keine komplexe Fahrzeugdynamik
- + **Für langsames Fahren** (Parken, Rangieren) gut geeignet (vorwiegend Geometrie relevant)
- + Kann mit Dubins-Heuristik effizient gemacht werden

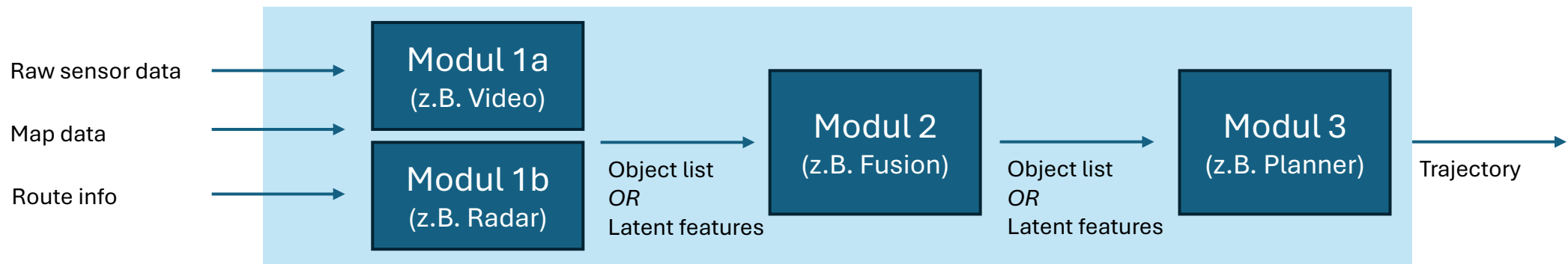
Nachteile:

- Teuer in dichter Umgebung (hochaufgelöstes Grid)
- Plant nur Geometrie, keine Zeit oder dynamischen Limits
- Für sehr dynamische Szenen **nicht geeignet**, z.B. Autobahn
- Der erhaltene Pfad muss oft nachträglich geglättet werden

Hybrid A*: Noch Stand der Technik im Bereich ADAS*?

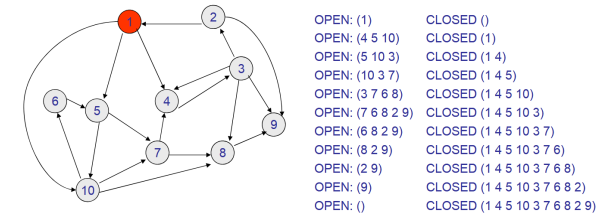
*Advanced Driver Assistance System

- Im Niedriggeschwindigkeitsbereich (z.B. Parken) ja
- Bei dynamischen Szenen weniger (ggf. als Fallback-Lösung)
- Alternativen dort (teils in Kombination mit)
 - **Sampling-basiertes Planen**: Berücksichtigung Fahrphysik, nicht deterministisch
 - **State-Lattice-Planner**: Vorab berechnete Motion Primitives, Suche im Lattice nach Optimum
 - (Non-linear) **Model-Predictive Control**: Start von initialem Pfad, Kostenoptimierung
 - *Zunehmend*: **Machine Learning**-basierte Methoden, bis hin zur Ende-zu-Ende Lösung:



Alternative zu A*: Greedy Best First Search

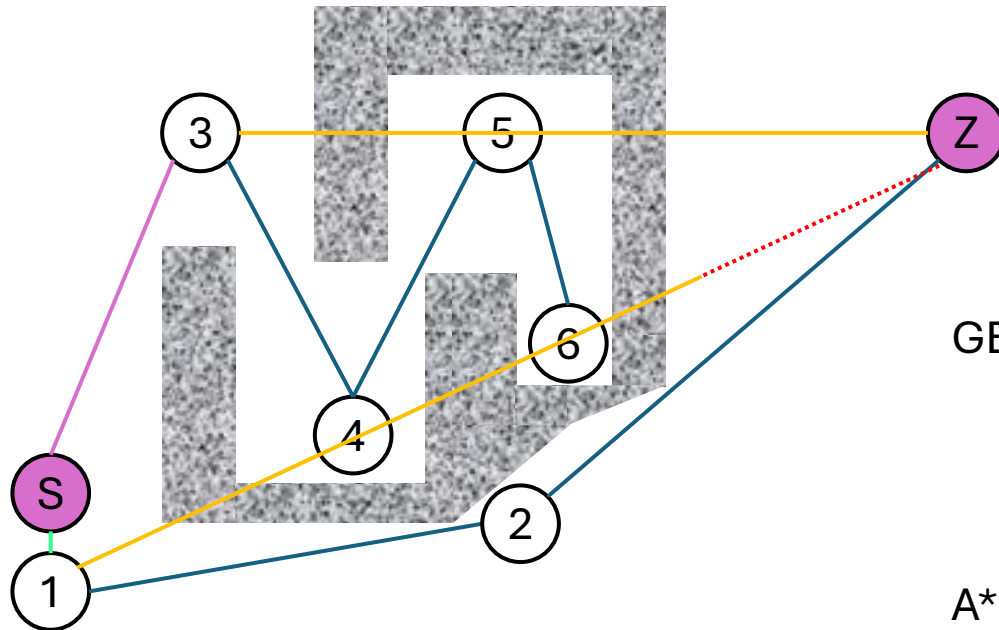
- Ergänzt ebenfalls die uninformierte Breitensuche um eine **Bewertungsfunktion h** für unexpandierte Knoten, z.B. „*welcher Knoten ist wahrscheinlich am nächsten zum Zielknoten*“
- Die OPEN-Liste wird gemäß dieser Funktion **sortiert**
- **Anders als bei A*** spielen die bisherigen Kosten *keine* Rolle



	A*	Greedy Best First
Prioritätsberechnung	$prio(v) = cost(v) + h(v)$ $cost(v)$: Kosten, um v zu erreichen $h(v)$: Heuristik für Kosten von v zum Ziel Kombination aus lowest-cost-first und best-first	$prio(v) = h(v)$
Eigenschaften	Balanciert Kosten und Zielschätzung	Gierig, schaut nur auf Heuristik
Optimal	Ja, wenn h „zulässig“ ist	Nein, kann lange Umwege machen
Schnelligkeit	Etwas langsamer, aber sicher optimal	Oft schneller, aber unzuverlässig

Alternative: Greedy Best-First Search

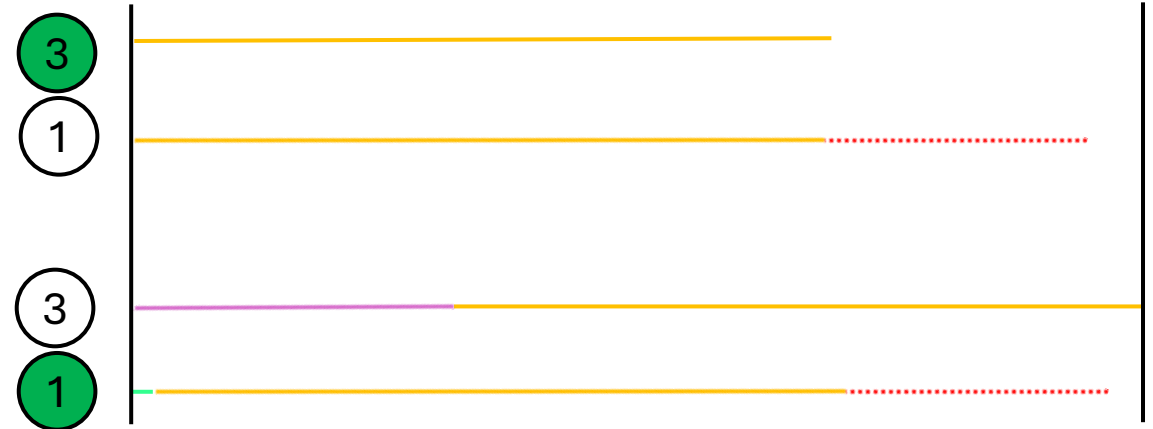
- Konstruiertes Beispiel, bei dem Greedy Best-First Search schlecht funktioniert, A* dagegen zu einem guten Ergebnis führt



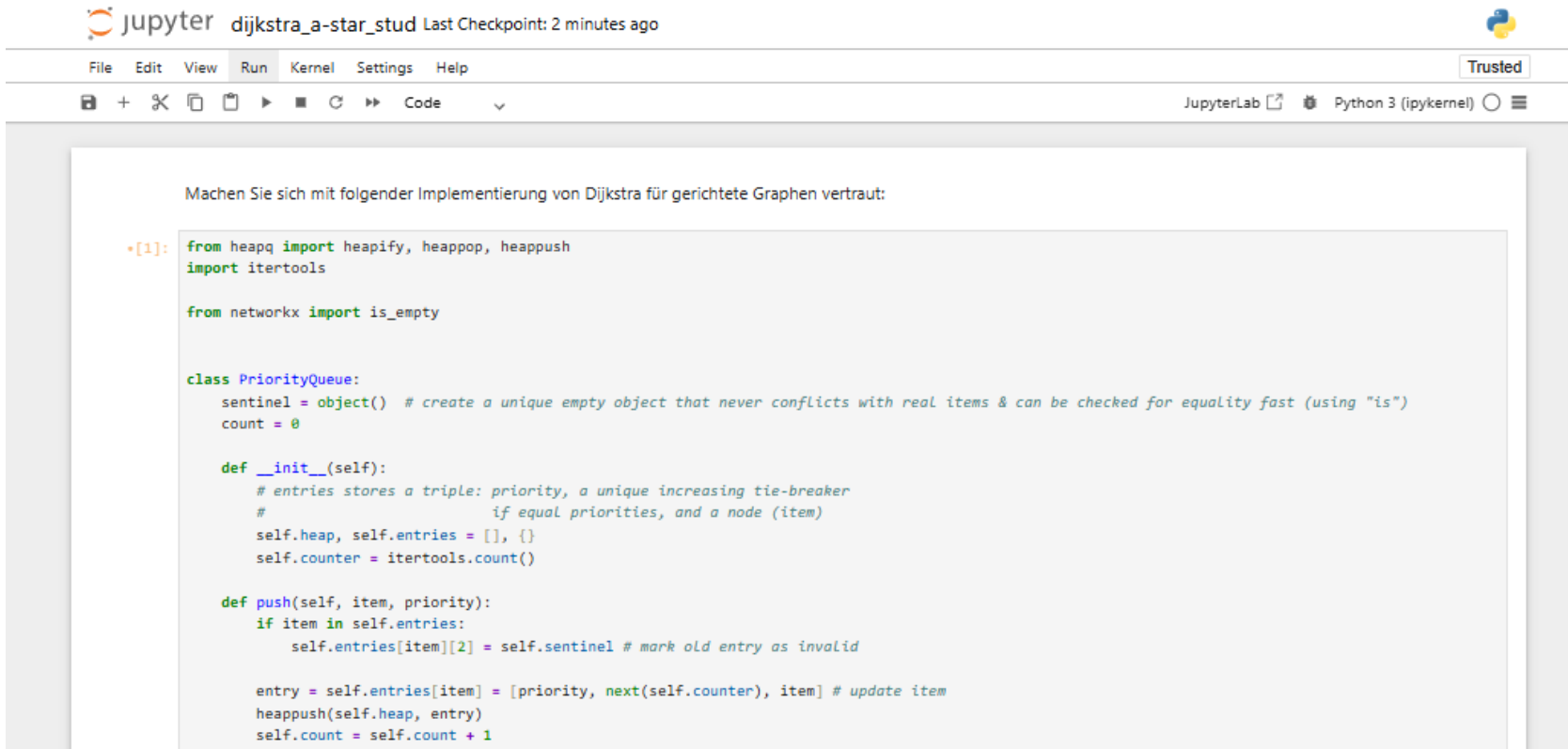
Annahmen:

- $h(v) = \text{Luftlinie}(v, Z)$
- Reale Kantenkosten entsprechen der Luftlinie

GBFS:



Jupyter Aufgabe zu A*



The screenshot shows a JupyterLab window titled "dijkstra_a-star_stud" with a "Last Checkpoint: 2 minutes ago" status. The interface includes a top menu bar with "File", "Edit", "View", "Run", "Kernel", "Settings", and "Help". A "Trusted" badge is visible on the right. Below the menu is a toolbar with icons for file operations and execution. The main area contains a code cell with the following Python code:

```
[1]: from heapq import heapify, heappop, heappush
import itertools

from networkx import is_empty

class PriorityQueue:
    sentinel = object() # create a unique empty object that never conflicts with real items & can be checked for equality fast (using "is")
    count = 0

    def __init__(self):
        # entries stores a triple: priority, a unique increasing tie-breaker
        # if equal priorities, and a node (item)
        self.heap, self.entries = [], {}
        self.counter = itertools.count()

    def push(self, item, priority):
        if item in self.entries:
            self.entries[item][2] = self.sentinel # mark old entry as invalid

        entry = self.entries[item] = [priority, next(self.counter), item] # update item
        heappush(self.heap, entry)
        self.count = self.count + 1
```

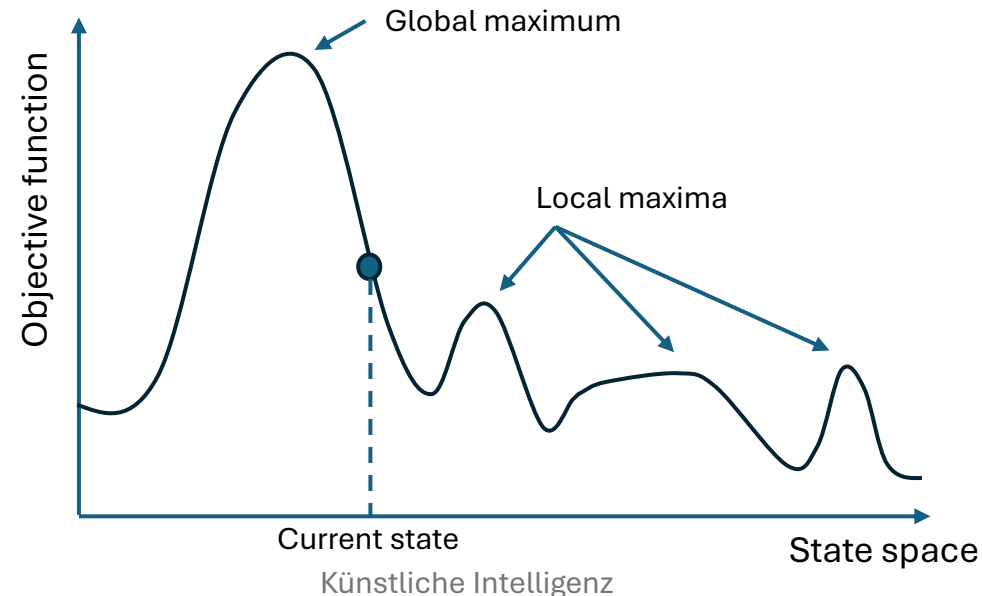
Suchverfahren in der KI

Teil 3: Ein kurzer Blick auf lokale Suchverfahren



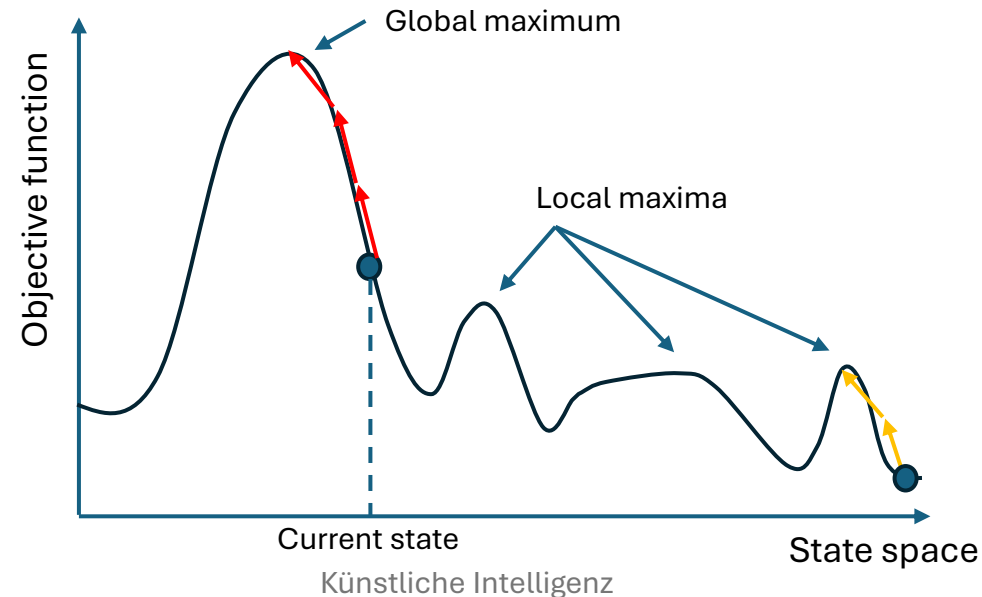
Einführung

- Bislang haben wir uns auf das Finden eines optimalen Pfades fokussiert
- Häufig ist das primäre Ziel jedoch, den **besten Zustand**, d.h. den Zustand mit dem maximalen oder minimalen Wert bezüglich einer Zielfunktion zu **finden**, der Pfad an sich ist nicht interessant
- Hier kommen lokale Suchverfahren wie **Hill Climbing** oder **Simulated Annealing** zum Einsatz



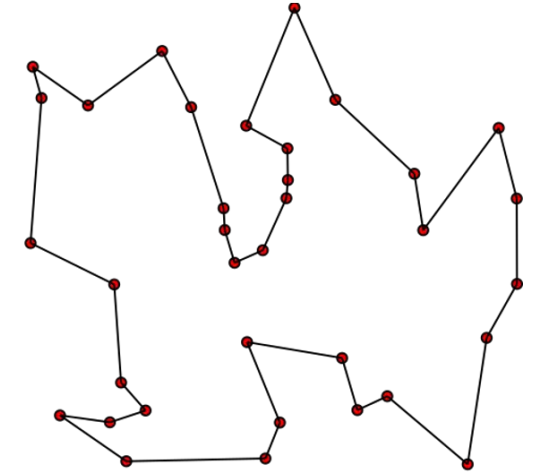
Hill Climbing

- Einfache **lokale Suche**:
 - Starte in einem beliebigen Zustand
 - Wähle iterativ und gierig (“greedy”) den Nachbarn mit dem besten Zielfunktionswert bis keine weitere Verbesserung möglich
- Kann in **lokalen Optima** hängenbleiben
- Hält immer nur einen Zustand im Speicher, **Startzustand** ist entscheidend



Hill Climbing – Anwendungsbeispiel: Traveling Salesman

- **Ziel** ist es, die kürzeste Route zu finden, auf der jeder Ort *genau ein mal* besucht wird und der Handelsreisende wieder *zum Startpunkt zurückkehrt*
- **Möglicher Lösungsansatz:**
 - als **Zielfunktion** fungiert die gesamte zurückgelegte Distanz
 - Beginne mit einer zufälligen Reihenfolge der Städte, z.B.
 $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$
 - Generiere mehrere mögliche **Nachfolgeknoten** durch Vertauschen (Swap) von je zwei Städten der Tour mit anschließender Berechnung der Zielfunktion (hier: Gesamtdistanz)
 - Wähle den **Nachfolger** mit der besten Bewertung
 - **Abbruch**, wenn keine weitere Verbesserung mehr erzielt wird oder Abbruchkriterium erreicht
 - *Hinweis:* Skaliert nicht gut, da es viele lokale Optima gibt



Hill Climbing – Anwendungsbeispiel: Traveling Salesman

- **Nachteil Swap:**

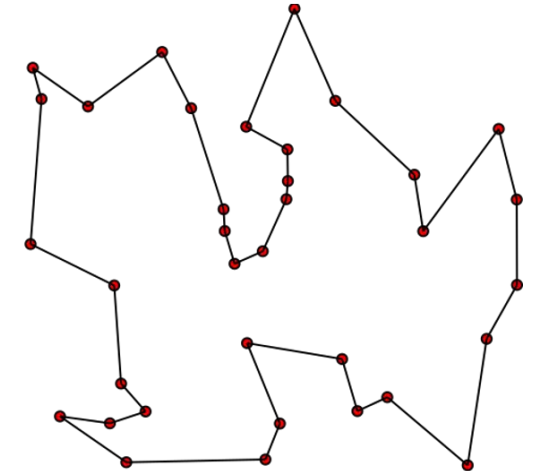
- Bis zu 4 Kanten ändern sich (z.B. bei Tausch von C und E)
- Mehr Sprünge im Suchraum, weniger lokal

- **Alternative: 2-Opt (oder 3-Opt):**

- *Gegeben:* Tour $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$
- Wähle (zufällig) **zwei Kanten** aus, z.B. $B \rightarrow C$ und $E \rightarrow A$
- Schneide diese Kanten aus, wir erhalten die Segmente
 $A \rightarrow B \mid C \rightarrow D \rightarrow E \mid A$
- Drehe den Teil zwischen den Schnittpunkten um und verbinde neu:
 $A \rightarrow B \rightarrow E \rightarrow D \rightarrow C \rightarrow A$
- Berechne die Länge der neuen Tour: *Wenn kürzer:* übernehmen

- **Hinweis:**

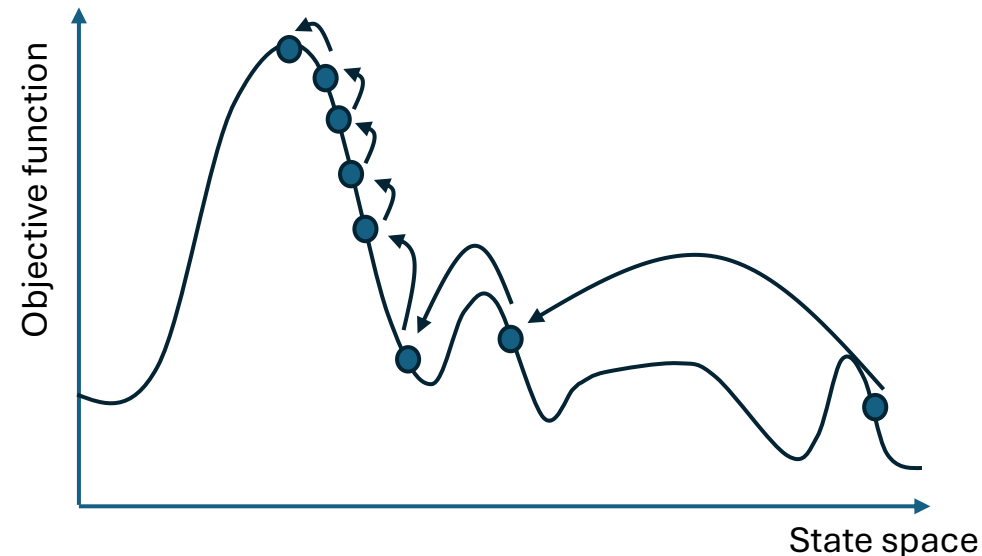
- 2-Opt alleine liefert Lösungen, die etwa 5-10% über dem Optimum liegen*, daher meist in Kombination eingesetzt (z.B. erst 2-Opt, dann 3-Opt für Verfeinerung)



* Johnson, David S., & McGeoch, Lyle A. (1997). The Traveling Salesman Problem: A Case Study in Local Optimization. Local Search in Combinatorial Optimization (S. 215–310)

Simulated Annealing („Simuliertes Abkühlen“)

- **Hill Climbing** akzeptiert nur Verbesserungen, bleibt sonst “stecken”
- **Simulated Annealing** akzeptiert mit gewisser Wahrscheinlichkeit auch Verschlechterungen, in der Hoffnung, dass man so lokale Minima überwinden kann
 - Wahrscheinlichkeit $\exp(-\Delta E / T)$, wobei ΔE die akzeptierte Verschlechterung und T ein Temperaturparameter ist, der über die Zeit gesenkt wird
 - Hohes T zu Beginn: viele Sprünge im Suchraum, niedriges T am Ende: Stabilisierung, nähert sich Hill Climbing



Zusammenfassung

- **Wiederholung**
 - Uninformierte Suche
 - Dijkstra
- **Informierte Suche**
 - A* und Hybrid A*
 - Greedy Best-First Search
- **Lokale Suche**
- **Anwendungsbeispiele**

